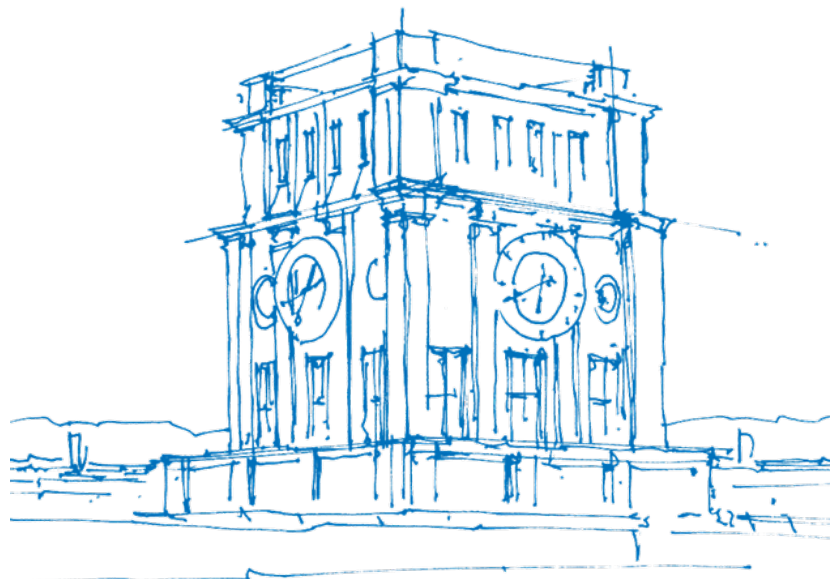


Master Thesis in Information Systems

Philipp Eichhorn

DeepBug: An Explainability Framework for Diagnosing and Improving Predictive Process Monitoring Models



TUM Uhrenturm

Master Thesis in Information Systems

Philipp Eichhorn

DeepBug: An Explainability Framework for Diagnosing and Improving Predictive Process Monitoring Models

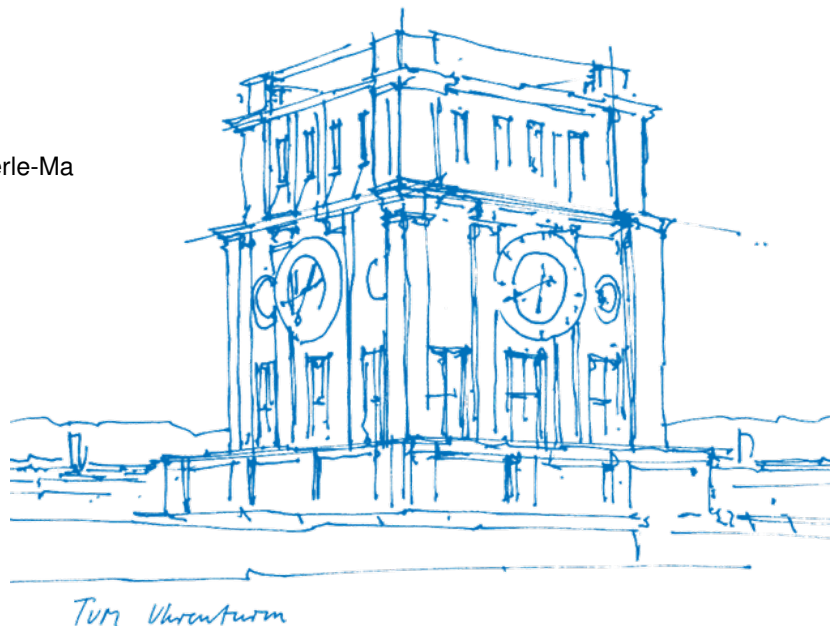
Thesis for the Attainment of the Degree
Master of Science

at the TUM School of Computation, Information and Technology,
Department of Computer Science,
Chair of Information Systems and Business Process Management (i17)

Examiner
Prof. Dr. Stefanie Rinderle-Ma

Supervised by
Henryk Mustroph

Submitted on
15.05.2026



Declaration of Academic Integrity

I confirm that this thesis is my own work and I have documented all sources and material used.

I am aware that the thesis in digital form can be examined for the use of unauthorized aid and in order to determine whether the thesis as a whole or parts incorporated in it may be deemed as plagiarism. For the comparison of my work with existing sources I agree that it shall be entered in a database where it shall also remain after examination, to enable comparison with future theses submitted. Further rights of reproduction and usage, however, are not granted here.

This thesis was not previously presented to another examination board and has not been published.

Garching, 15.05.2026

Philipp Eichhorn

Contents

1	Introduction	7
1.1	Motivation	7
1.2	Research Questions	8
1.3	Evaluation	8
2	Preliminaries	9
2.1	Explainability and Interpretability	9
2.2	Predictive Process Monitoring (PPM)	10
3	Related Work	11
3.1	Intrinsically Interpretable Models	13
3.2	Review Methodology	13
3.3	Post-Hoc Explainability in PPM	14
3.4	Visualization	21
3.5	Diagnostic Use of Explanations	21
3.6	Decision Mining	22
3.7	Summary and Research Gap	23
4	Datasets	23
4.1	Helpdesk	24
4.2	BPIC-17	24
4.3	Sepsis	25
4.4	Domestic Declarations	25
5	Models	25
5.1	Uncertainty-aware Encoder-Decoder LSTM	25
5.2	Camargo et al.'s LSTM	26
6	Explainable AI for Suffix Predictors	28
6.1	Attribution Scores	28
6.2	Process Model-Based Visualization	34

	4
6.3 Gateway Analysis	37
7 Analysis	40
7.1 Uncertainty-aware Encoder-Decoder LSTM (U-ED-LSTM)	40
7.2 Camargo et al.'s LSTM	51
8 Framework	59
8.1 Data Set Level Aggregation	60
8.2 Process Prediction Map	61
8.3 Gateway Surrogates	63
8.4 Uncertainty-aware Encoder-Decoder LSTM (U-ED-LSTM)	65
8.5 Camargo et al.'s LSTM	70
9 Discussion	74
9.1 Findings	74
9.2 Limitations	75
9.3 Future Work	76
10 Conclusion	77
Bibliography	78
A Appendix	87

List of Tables

1 Overview of XAI Methods Applied in the PPM Domain	15
2 Configurations for U-ED-LSTM	26
3 Configurations for Camargo et al.'s LSTM	28
4 Per-gateway Accuracy of Camargo et al.'s LSTM, a Ground-truth Decision Tree and a Surrogate Model	59
5 Framework Signals and Model in which they Appear	65
6 Prediction Accuracy per Gateway Before and After Adjustments	73
7 Accuracy across U-ED-LSTM and Camargo et al.'s LSTM	74
8 Engineered features used by the per-gateway surrogate decision trees (BPIC17)	88

9	Engineered features used by the per-gateway surrogate decision trees (Helpdesk) . .	89
---	---	----

List of Figures

1	Taxonomy of XAI methods	12
2	Next-event Attribution Heatmap for a BPIC17 Prefix	32
3	Full Suffix Attribution Heatmap for a Helpdesk Prefix	33
4	Mean Absolute Attribution Scores of U-ED-LSTM on BPIC-17	34
5	Mean Aggregate Integrated Gradient at Gateway "A_Create Application" of U-ED-LSTM for BPIC-17	35
6	Process Prediction Map of U-ED-LSTM for Helpdesk	37
7	Process Prediction Map of U-ED-LSTM for BPIC-17	37
8	Example of a Ground Truth Decision Tree and Gateway-specific Surrogate Tree . .	40
9	Process Prediction Map of U-ED-LSTM on Helpdesk	41
10	Next Event Integrated Gradient Score of Example Case	42
11	Attribution Scores (IG and SHAP) for Sample Prediction	43
12	Aggregated Integrated Gradients of U-ED-LSTM on Helpdesk	44
13	Aggregated Integrated Gradients of U-ED-LSTM on BPIC-17	45
14	Process Prediction Map of U-ED-LSTM on BPIC-17	46
15	Decision Trees for U-ED-LSTM on BPIC-17 at Gateway "A_Create Application" .	46
16	Mean Aggregate Integrated Gradient at Gateway "A_Create Application" of U-ED-LSTM on BPIC-17	47
17	Decision Trees on U-ED-LSTM for BPIC-17 at Gateway "W_Handle Leads" . . .	48
18	Aggregated Integrated Gradients of U-ED-LSTM for Sepsis	49
19	Process Prediction Map for U-ED-LSTM on Sepsis	50
20	Decision Trees on U-ED-LSTM for Sepsis at Gateway "Admission NC"	50
21	Process Prediction Map for U-ED-LSTM on Domestic Declarations	51
22	Aggregated Integrated Gradients of Camargo et al.'s LSTM for Helpdesk	52
23	Process Prediction Map for Camargo et al.'s LSTM on Helpdesk	52
24	Aggregated Integrated Gradients of Camargo et al.'s LSTM for BPIC-17	54
25	Process Prediction Map of Camargo et al.'s LSTM on BPIC-17	55
26	Per Gateway Aggregated Feature Attribution Scores of Camargo et al.'s LSTM for Sepsis	56

27	Process Prediction Map of Camargo et al.'s LSTM on Sepsis	57
28	Decision Trees for Camargo et al.'s LSTM on Sepsis at Gateway "Admission NC" .	58
29	Process Prediction Map for Camargo et al.'s LSTM on Domestic Declarations . . .	60
30	The Three Steps of the DeepBug Framework	60

Abstract

Predicting subsequent events of an ongoing process is one of the core challenges in the Predictive Process Monitoring (PPM) field. Enabling users and developers of such PPM models to gain a deep understanding of why the model makes a given prediction is a key challenge to improve performance and adoption of the systems. This thesis introduces **DeepBug**, a framework that enables a user to make in-depth analysis of their PPM models to diagnose and improve their models. This framework builds on classical attribution methods (Integrated Gradients and SHAP), introduces the *process prediction map*, a directly-follows graph mined from the event log and overlaid with the model’s predictions, and *per-gateway surrogate decision trees* which draw on decision-mining practices to surface the model’s learned branching mechanism. The framework itself provides a three-step mechanism (Observation, Diagnosis, Action) to derive model enhancements from patterns surfaced by the methods. We evaluate the framework based on two suffix-prediction architectures and across four event-log datasets. In all seven signal applications the framework uncovers diagnostic patterns, and in two of those it yields accuracy improvements.

1 Introduction

1.1 Motivation

The field of process mining has long provided valuable insights into the inner workings of processes through tools and methodologies designed for post-hoc analysis of event logs. A key innovation now lies in extending these retrospective capabilities toward generating actionable insights for ongoing cases [1], which enables proactive interventions in live process instances with predictive process monitoring (PPM). In recent years, the field has shifted from intrinsically interpretable models such as decision trees to more complex black-box models and deep neural networks. While this shift has improved predictive performance, it has also reduced interpretability and increased the need for post-hoc explainability methods to understand model behavior for two main reasons: Firstly, it provides the user transparency in the models decision-making, which is important for users to accept the predictions and use the system. Secondly, it helps researchers understand model behavior and limitations, which in turn supports the development of more accurate and reliable predictive models. Increasing trust in the system is supported by both transparency and accuracy, and thus by

explainability. While both audiences benefit from explainability, this thesis focuses on the developer use case and investigates how they can leverage explainability to improve their models.

1.2 Research Questions

RQ 1: What state-of-the-art explainability methods can be applied to PPM?

This question provides an overview of explainability methods commonly used in predictive process monitoring, as well as methods adopted from neighboring fields. The goal is to identify which techniques are suitable for explaining the behavior of event prediction models in PPM.

RQ 2: How can the identified explainability methods be best implemented for PPM and how can artifacts from the process domain be used to explain model behavior?

This part focuses on the implementation of the selected explainability methods and the development of new ones with the goal of generating meaningful explanations of a PPM model’s behavior and its event predictions.

RQ 3: Can those explanations be used to derive means to achieve model improvements?

In this part, we evaluate to what extent the implemented explainability methods support model improvement.

1.3 Evaluation

The evaluation of explainability methods has been a long-standing challenge in explainability research due to the inherently subjective nature of explanations. Whether something is perceived as “explainable” depends on characteristics of the person receiving it. Factors such as domain knowledge, the ability to process complex information, and familiarity with the explanation pattern all influence how well an explanation is understood. To provide an evaluation that focuses on the capabilities of the method rather than on the characteristics of the recipient, we assess the explainability methods based on their ability to achieve model improvement. They are evaluated by being applied in a structured way as part of the DeepBug Framework that uses a three-step procedure (Observation, Diagnosis, Action) to diagnose issues and improve the model’s accuracy. First, we analyse what information the explanation methods provide and detect patterns that indicate issues. Second, we

hypothesise what the most likely cause for those patterns is. Third, we derive and implement suitable model enhancements based on the likely cause to address and verify if the underlying cause could indeed have been resolved. By applying this framework to two models across three datasets, we evaluate if the framework, and by extension the underlying explainability methods, can indeed drive model improvements.

2 Preliminaries

2.1 Explainability and Interpretability

The definitions of *explainability* and *interpretability* vary significantly across the literature and are mostly used interchangeably, describing an understanding of the underlying model that goes beyond a single input-output pair or an accuracy metric. The goal of achieving a clear understanding of what the terms mean and how they differ has been attempted by a few researchers in the field, and from these attempts two broad positions have emerged, differing in whether explainability is treated as a property of the model itself or as a property of a human’s understanding of it. Some works adopt philosophically grounded perspectives, such as [2], who argue that neural networks are in principle explainable, since their computations are deterministic and their mathematical reasoning, e.g., through open model weights, is accessible, regardless of whether a human can practically extract this information. Others, such as [3], take a more human-centered stance, requiring models or methods to produce explanations that are teachable to humans in order to be deemed explainable. Further, she argues that the model can only really be understood by a human if it is intrinsically interpretable. [3] further differentiates between explainability, which seeks to describe the behavior of a black-box model post hoc, and interpretability, which aims to make the model itself transparent by construction. In contrast, standards such as the ISO/IEC 42001 [4] do not draw a strict distinction between the two terms, instead viewing interpretability as an aspect of explainability. To avoid ambiguity and maintain focus, this thesis adopts the terminology most commonly used in current research: the term explainability will be used as an umbrella concept. When referring to models that are inherently understandable without external explanatory methods, the prefix *intrinsic* will be added (i.e., *intrinsic interpretability*). This approach ensures conceptual clarity and allows readers to intuitively grasp the intended meaning while keeping our definitions concise. Based on the above, the following working definitions are adopted throughout this thesis.

(Post-hoc) Explainability. A model’s explainability describes the extent to which a human can understand why the model produced a particular prediction (local explainability) or how the model behaves in general (global explainability). Explainability is, therefore, a continuous property that can be enhanced through the application of explainability methods. The perceived level of explainability may vary depending on the observer’s familiarity with the model and its domain.

(Intrinsic) Interpretability. Intrinsic interpretability describes models that provide transparent reasoning for their predictions, which can be fully understood by a human without the application of additional explanatory methods.

Explainability Method. An explainability method is a procedure that can be applied to machine learning models to provide insight into why the model made a certain prediction (local) or how the model makes predictions in general (global).

2.2 Predictive Process Monitoring (PPM)

The concept of predictive process monitoring (PPM) emerged from the broader field of process mining, with the aim of extending traditional, retrospective analyses toward the prediction of future behavior in ongoing process instances. The need for such predictive capabilities was first mentioned in the Process Mining Manifesto by [1], which emphasized the potential of pushing process mining past its descriptive application areas to include proactive capabilities as well. Building on this vision, [5] formally introduced predictive process monitoring as a means to predict future attributes of a running case, using historical process execution data. In contrast to classical process mining techniques, which focus on discovering, checking, or enhancing completed process models, PPM enables organizations to anticipate future events and intervene in real time, thereby significantly expanding the scope and impact of process mining applications. As a result, predictive process monitoring has evolved into one of the prominent and dynamic sub-fields within the process mining domain.

Next Event Prediction Next event prediction refers to the task of forecasting the immediate next activity of an ongoing process execution. Instead of predicting the full continuation of the case, the model returns only the most likely next event, often including any relevant attributes such as activity type or predicted timestamp. This task is commonly used as a fundamental building block for more

complex predictive tasks, such as suffix prediction, and often serves as a baseline for evaluating the predictive capabilities of process monitoring models.

Suffix Prediction Suffix prediction defines the task within predictive process monitoring, which predicts the array of upcoming activities until and including the end of the predicted process trace. The activities themselves can be defined differently, but they usually include at least one categorical attribute (e.g., the activity type). Additionally, other attributes can be included in the definition of an activity, such as a continuous value (e.g., activity duration).

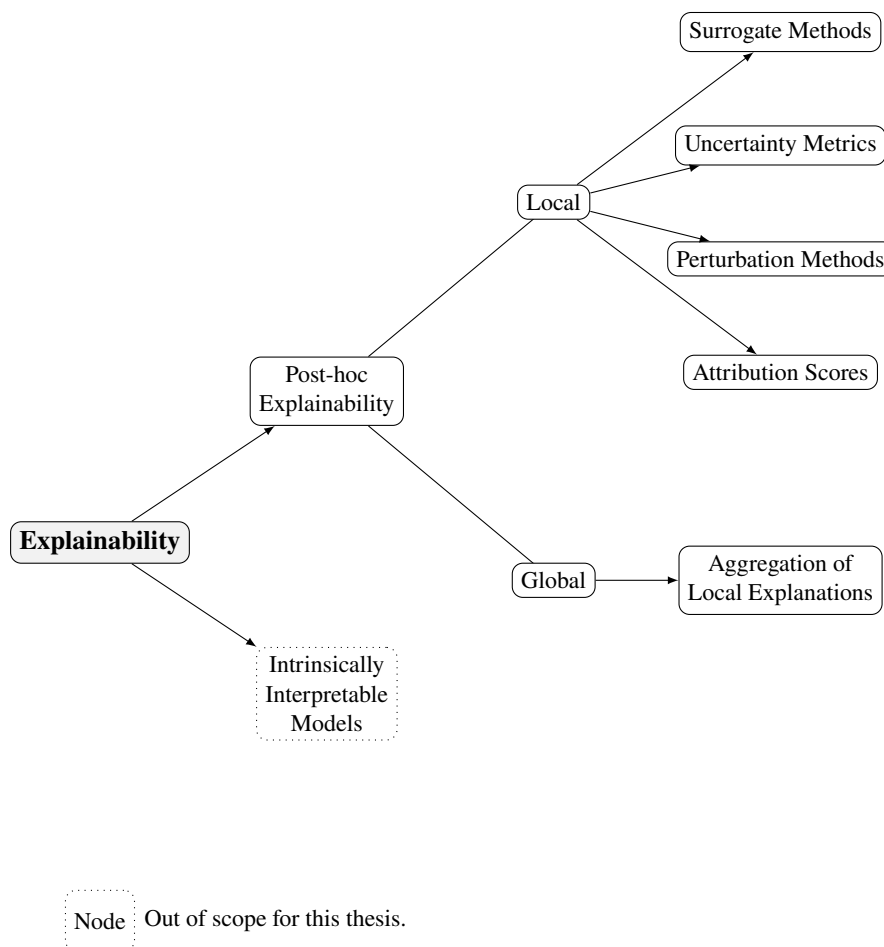
Outcome Prediction Outcome prediction focuses on forecasting whether an ongoing case will result in a certain predefined outcome. These outcomes are typically formulated as binary or multi-class labels, such as whether a case will violate a deadline, result in non-compliance, or achieve a desired performance threshold. The model uses the currently observed prefix of a case to estimate the likelihood of each possible outcome. Compared to suffix prediction, outcome prediction provides a more aggregated view of the future, as it does not require the prediction of each individual upcoming activity.

Gateway We define gateways as points in the process where an activity is followed by different successor activities. Gateways are fully defined by the activity at which they occur, such that for a gateway at activity A, all process traces that contain an event with activity A pass through this gateway, and any prefix whose last event has activity A makes the corresponding gateway decision when its next event is predicted. While process models distinguish XOR and AND gateways, in PPM all gateways effectively appear as XOR as each trace only has one next entry and the prediction model cannot express parallelism either. Our gateway definition is therefore based on a data-driven, not process model-driven understanding of the process.

3 Related Work

This Chapter positions the contribution of this thesis within the existing body of research on explainability in PPM. After describing the literature search underlying the review, this Chapter

Figure 1
Taxonomy of XAI methods



discusses the position of intrinsically interpretable models, presents a structured overview of post-hoc explainability methods that have been applied or developed in the PPM space, and briefly discusses adjacent work on decision mining.

In the following, an overview of explainability methods used in the PPM literature is provided, structured according to the taxonomy shown in Figure 1. At the top level, methods are divided into intrinsically interpretable models and post-hoc explainability methods. Post-hoc methods are further split into those providing local explanations, i.e., explanations for a single prediction, and those providing global explanations, i.e., explanations of the model’s general behavior. The scope of this Chapter, and of the thesis as a whole, is restricted to post-hoc methods, for reasons discussed below.

3.1 Intrinsically Interpretable Models

Intrinsically interpretable models have a long tradition in XAI research, and some authors argue that, in many settings, such models can match the performance of black-box models [3]. In PPM, however, the rise of deep learning has moved the practical state of the art towards black-box architectures, which now dominate the recent literature on activity, outcome, and suffix prediction [6]–[20]. Since this thesis builds on such a deep-learning model, the remainder of this Chapter focuses on post-hoc explainability, which applies to black-box architectures regardless of their internal structure.

3.2 Review Methodology

The review in this Chapter is based on a structured literature search aimed at situating this thesis within the existing work on explainability in PPM.

Databases and time range Three databases were queried: Web of Science, Scopus, and IEEE Xplore. Web of Science and Scopus were chosen because they cover both the journals and the conference proceedings relevant to PPM; IEEE Xplore was added to catch conference venues that the other two sometimes miss. No publication-year restriction was applied at the query level, since the fields are relatively new.

Query The query combined explainability terms with PPM terms using a Boolean AND. The explainability block covered the common names of the subfield and its main method families; the PPM block covered the prediction targets studied in the literature. The full Scopus query, representative of the corresponding queries on Web of Science and IEEE Xplore, was:

```
(TITLE-ABS-KEY(Explainability OR Interpretability OR "Explainable
AI" OR XAI OR "Model Explanation" OR "Model Interpretation" OR "Model
Transparency" OR "Post-hoc Explanation" OR "Feature Importance" OR
"Attention Interpretation"))
AND
(TITLE-ABS-KEY("Predictive Process Monitoring" OR "Predictive Business
Process Monitoring" OR "Process Outcome Prediction" OR "Process Prediction"
OR "Next Event Prediction" OR "Remaining Time Prediction" OR "Process
```

```
Mining Prediction" OR ("Predictive Analytics" OR "Process Mining")
OR "Event Log Prediction" OR "Process Instance Prediction"))
```

This query returned 55 results on Web of Science, 90 on Scopus, and 13 on IEEE Xplore. After cross-database de-duplication, 91 unique publications remained as the starting set for screening.

Inclusion and exclusion criteria From this combined set, publications were kept if they addressed the application or development of explainability methods in a PPM context, inclusive of all tasks: outcome prediction, next-activity prediction, remaining-time prediction, and suffix prediction. Pure literature surveys were excluded and surveys were kept aside for orientation rather than used as primary references. Also excluded were papers in which the link to process logs or to a prediction task was not clear.

Resulting Set The resulting set underpins the taxonomy in Figure 1. The remainder of this Chapter is organized along this taxonomy, so that the reader can place each method within the part of the explainability space that is relevant for PPM.

3.3 Post-Hoc Explainability in PPM

Post-hoc explainability methods are split in this Chapter into local methods, which aim to explain single prediction outcomes and global methods, which explain the behavior of the model as a whole. Within the local methods, we distinguish between four types of methods: uncertainty quantification, surrogate methods, feature attribution, and counterfactual and perturbation-based methods. Within the global methods, three types are distinguished: representation analysis, aggregation of local methods and model distillation. Additionally, the research focusing on visualization of explanations which runs across the mentioned types of methods rather than being its own standalone category, is discussed separately at the end.

Local methods

Uncertainty Quantification. Uncertainty quantification methods explain a prediction by reporting how confident the model is in producing it. Although uncertainty quantification is sometimes treated as a separate concern from explainability, it is included here as an explainability

Table 1
Overview of XAI Methods Applied in the PPM Domain

Method	Category	Suffix Prediction	References
Bayesian Neural Networks	Uncertainty Quantification	yes	[21]
Monte-Carlo Dropout	Uncertainty Quantification	yes	[22], [21]
Conformal Prediction	Uncertainty Quantification	yes	[23]
SHAP	Feature Attribution	–	[24]
Individual Conditional Expectations	Feature Attribution	–	[24]
Attention-based explanations	Feature Attribution	–	[25]–[28]
LORELEY	Counterfactual and Perturbation-Based	–	[29]
REVISED+	Counterfactual and Perturbation-Based	–	[30]
PABLO	Surrogate Explanations	–	[31]
CA-IMPRESSED	Surrogate Explanations	–	[32]
SP-LIME	Aggregation of Local Explanations	–	[33]
Visual analytics for uncertainty	Visualisation	–	[34]
Business-oriented visualisation dashboard	Visualisation	–	[33]
Explanation style user study	Visualisation	–	[35]

category, since knowing how confident a model is in a particular prediction provides the user with additional information enhancing the interpretation of the prediction. Two complementary lines of work can be distinguished: methods that rely on components within the model’s architecture to produce probability estimates, and methods that wrap a trained model as a black box and infer uncertainty from its outputs alone.

An example of the architecture-dependent line are Bayesian Neural Networks, as used by [21]. In that case, the model outputs a probability distribution, from whose characteristics the certainty of the prediction can be inferred. A conceptually similar approach is Monte-Carlo Dropout, which disables a random subset of neurons during inference in order to obtain a set of predictions that collectively represent a distribution over the model’s output. [22] apply Monte-Carlo Dropout in the PPM setting and wrap its output in the conformal prediction framework, which is introduced below, in order to convert the dropout-based credible intervals into prediction intervals with formal coverage guarantees. The result is a hybrid approach: Monte-Carlo Dropout is architecture-dependent, while the conformal prediction wrapper is model-agnostic. [22] also visualize the resulting uncertainty information by borrowing techniques from other explainability families, such as Individual Conditional Expectation and Partial Dependence Plots.

An example of the model-agnostic line is the work of [23], who apply conformal prediction directly to a trained black-box classifier in a healthcare PPM setting, without any architectural intervention in the model itself. Conformal prediction is a post-hoc technique that takes the outputs of any trained classifier and turns them into prediction regions with a guaranteed coverage level: if the target coverage is set to, say, 95%, the region produced by conformal prediction is guaranteed to contain the true outcome in at least 95% of cases. The coverage guarantee, however, depends on the classifier’s output probabilities being well-calibrated to begin with, which is rarely the case for deep models out of the box. [23] therefore study how a range of post-hoc probability calibration techniques affect the validity and tightness of the resulting prediction regions when they are fed through conformal prediction. How such uncertainty estimates are subsequently visualized is itself treated as an explainability problem, as discussed in Section 3.4.

Surrogate Explanations. Surrogate explanations explain a prediction by training a simple, intrinsically interpretable model that approximates the black-box model’s behavior around the prediction of interest. The explanation is then phrased in terms of the surrogate’s structure, such as

a decision tree path or a set of rules. In the PPM literature, the methods that fall into this category either mine recurrent control-flow or context patterns from the underlying event log and relate them to the model’s output, or define local neighborhoods in the model’s internal representation space and fit a surrogate for each neighborhood.

PABLO (Pattern-Based Local Explanations) [31] generates two sets of neighborhood traces around the case to be explained: factual traces, which the black-box model classifies the same way as the inquiry case, and counterfactual traces, where the classification flips to a different outcome. It then identifies recurrent control-flow patterns whose presence separates the two sets. Patterns that occur in the counterfactual neighborhood but not in the factual one are taken as genuine drivers of the prediction, which distinguishes them from patterns that are merely frequent. The resulting patterns are then presented as the explanation. CA-IMPresseD [32] extends the IMPresseD pattern-mining framework along a different axis, by integrating multiple process perspectives into the patterns it discovers. Where earlier methods such as LORELEY and PABLO consider the control-flow perspective only, CA-IMPresseD also incorporates resource, time, and data attributes into the patterns, which makes the resulting explanations sensitive to context information that purely control-flow methods miss.

A different approach within the same category was proposed by [36]. Rather than mining process patterns, their method exploits the model’s internal representations to define local neighborhoods: the intermediate latent-space representations from the last hidden layer of the network are clustered using k-means, and a separate local surrogate decision tree is trained for each cluster. The use of latent-space proximity as the neighborhood criterion contrasts with perturbation-based methods such as LIME, where the neighborhood is generated by sampling around the input.

Feature Attribution. Feature attribution methods assign a score to each input feature, quantifying how much that feature contributed to a particular prediction. Within PPM, four classes of feature attribution are relevant here, three of which have been directly applied in prior work. SHAP (SHapley Additive exPlanations) [37], based on cooperative-game-theoretic Shapley values, has been applied to PPM tasks by [24], using the permutation-based Shapley variant of [38] via the `iml` R package. In the same work, Individual Conditional Expectation (ICE) plots are used to visualize how the prediction for a single instance changes as one feature is varied while all others are

held fixed. The resulting curves show the local response function of the model around each instance and can be read as a visual form of sensitivity analysis.

Attention weights form a third attribution signal, available by construction in any model with a built-in attention mechanism. They have been adopted for PPM both through post-hoc explanation methods derived from a trained model’s attention scores [27], [28] and through specialized attention mechanisms designed into the prediction architecture for built-in interpretability [25], [26]. The use of attention weights as a faithful explanation has been subject to debate in the broader XAI literature [39], [40], but they remain a commonly adopted signal in the PPM context.

A further class of attribution methods are gradient-based methods, which assign importance to input features using derivatives of the model’s output with respect to the input. Simple variants, such as the raw gradient or Gradient $\times$ Input, produce noisy attributions and fail to satisfy basic attribution axioms such as sensitivity [41]. Integrated Gradients [41] addresses these shortcomings by integrating the gradient of the output along a straight path from a reference baseline to the actual input, and is the first attribution method to provably satisfy both the sensitivity and the implementation invariance axioms introduced in the same paper. In practice, Integrated Gradients needs only the standard gradient operator and no architectural modification. In the PPM literature, gradient- and backpropagation-based attribution has so far been explored primarily through Layer-wise Relevance Propagation (LRP), which backpropagates the model’s output relevance through the network to attribute it to individual input features. [42] apply LRP to LSTM-based next activity prediction. Integrated Gradients itself has, to the best of our knowledge, not been applied to PPM.

Counterfactual and Perturbation-Based Explanations. Counterfactual and perturbation-based methods explain a prediction by asking what would have needed to change in the input for the model’s output to be different. The two types are closely related: perturbation-based methods systematically alter parts of the input and observe the effect on the output, while counterfactual methods aim to identify the smallest such alteration that flips the prediction. Many surrogate-based explainability methods, such as LIME and LORE, also rely on perturbations as their underlying mechanism.

In PPM, the main challenge for both types is that perturbed inputs need to remain valid process traces, since arbitrary perturbations of an event sequence may produce traces that the underlying process

could never generate. Two approaches have targeted this validity requirement directly: LORELEY [29], an extension of LORE designed for process outcome prediction, treats the control flow, that is, the prefix, as a single attribute when generating perturbations to train a decision-tree surrogate. As a result, all perturbations are valid activity sequences, from which counterfactual explanations can subsequently be extracted. REVISED+ [30], also developed for outcome prediction, applies two constraints during counterfactual generation: it restricts the search to a high-density region of the observed traces, so that the generated counterfactuals lie within the observed data distribution, and it uses Declare language templates [43] to enforce sequential patterns that are consistent with the historical cases. Declare is a declarative process-modeling language in which a process is specified through the constraints that a valid trace must satisfy, for example that activity B must eventually follow activity A, or that activities A and B must not co-occur. The two approaches differ in how they ensure validity. LORELEY treats the prefix as a single categorical attribute when generating perturbations, so the generated activity sequences are valid by construction. REVISED+ instead performs two preparation steps that LORELEY does not need. First, a variational autoencoder is trained on the observed traces to obtain a low-dimensional manifold that defines the high-density region. Second, Declare templates mined from the event log are added as extra loss terms during the same training, so that the manifold respects the observed sequential patterns. REVISED+ has been applied only to full traces, since its Declare template mining requires complete cases to learn the process constraints. Counterfactual perturbations are then produced by encoding the original trace into the latent space of the variational autoencoder taking iterative gradient steps under a loss function that combines a class-flipping term, a proximity term, and a Declare-constraint term. At each step, the current latent point is decoded back into a candidate counterfactual trace.

Global Methods

Global methods describe the behavior of the model as a whole, rather than explaining individual predictions. In the PPM literature, three approaches have been followed: feature importance, which is the most common, global visualization of feature response, and aggregated local explanations projected onto the process model.

Feature Importance. Feature importance methods produce a ranking of input features by how much they influence the model’s predictions on average across a dataset. In PPM, three variants have been used.

The first is aggregation of local attribution scores. Given a local method such as SHAP, taking the mean absolute SHAP value for each feature across a test set produces a global feature-importance ranking that is built from the same per-instance SHAP values used for local explanations, simply averaged across the dataset. This is the standard SHAP-based global view [37] and has been applied to PPM by [44] for remaining-time and outcome prediction, and by [45] as part of a broader study of explainability for outcome prediction. The same aggregation also applies to LRP: averaging LRP relevance scores for each input variable across a test set yields a feature-importance ranking, an approach followed in the PPM context by [42] and [25], among others.

The second is permutation feature importance, in which the values of a single feature are shuffled across the dataset and the resulting drop in model performance is taken as a measure of that feature’s importance. This approach is model-agnostic and has been applied to outcome prediction in PPM by [46].

The third is model-specific feature importance, which reads importance directly from the structure of the trained model. For deep neural networks, [24] derive feature importance from the connection weights.

Global Visualization of Feature Response. A second line of global explanation work adds an additional dimension: instead of a single importance score per feature, it produces a curve showing how the model’s prediction responds as that feature’s value varies. Partial Dependence Plots (PDPs) compute this curve by varying the chosen feature across its range, re-scoring the whole dataset at each value, and averaging the resulting predictions; the curve shows the model’s average response to the feature. PDPs have been applied to PPM by [24]. Individual Conditional Expectation (ICE) plots perform the same computation without the averaging step, so each instance contributes one curve. This exposes variation across cases that PDPs hide by averaging. In the PPM literature, [24] also use ICE plots in this way, as a global view on feature response.

Aggregation with Process-Model Projection. A third approach goes beyond a feature-importance ranking by projecting the global view onto the underlying business process model. [33] build on SP-LIME [47], a method that uses submodular optimization to pick a small set of instances whose LIME explanations collectively cover the most influential features while minimizing redundancy. For an outcome-prediction classifier, they project the resulting aggregated explanations

onto the process model, so that each feature’s contribution to a prediction is visible in the process structure that users are already familiar with. Its visualization is discussed further in Section 3.4.

3.4 Visualization

The US Defense Advanced Research Projects Agency’s (DARPA) XAI program [48], launched in 2017, identified the design of effective explanation interfaces as one of its three pillars, alongside explainable models and the psychology of explanation, underlining the importance of visualization as part of explainability research. The motivation is grounded in one of the central purposes of explainable AI: to increase the end user’s trust in a system by providing additional information about why a particular prediction was made. Increased trust, in turn, supports the adoption of AI in high-risk fields such as medicine and defense.

Within PPM, three contributions stand out. [34] extend Partial Dependence and Individual Conditional Expectation plots with uncertainty information, so the same panel shows both how the model responds to a feature and how confident it is in that response, and evaluate the resulting dashboard through expert interviews on a manufacturing PPM use case. [33], discussed in Section 3.3.2 for the aggregation step it builds on, also treats the visualization itself as the contribution: the per-feature contributions returned by SP-LIME are projected onto the process model, and the approach is illustrated on a claim-management classifier with accept/reject outcomes. The novelty here is using the process model, a process-mining artifact, as the medium on which the explanation is read. [35] run a controlled user study that crosses three explanation styles feature importance, rule-based, and counterfactual with a high or low manipulated level of perceived model accuracy, and find that both factors significantly affect the objective measures (task performance and agreement with the model) and subjective decision confidence.

3.5 Diagnostic Use of Explanations

The explainability methods reviewed in Sections 3.3 and 3.4 are presented primarily as end-user-facing tools, that is, for domain experts who make a decision based on the model’s prediction. The use of explanations here is to enhance the resulting decision quality by providing the user with additional information about the model, the prediction, and how the model derived the prediction. However, a second use, which is established in broader machine learning research, focuses on the model developer as the user of the explanations. The goal is to uncover what the model has actually learned and derive model improvements from that.

Multiple papers show that explainability methods can uncover so-called shortcut learning, where a model solves the training task by latching onto a spurious feature in the data instead of the signal it was meant to learn [49]. The original LIME paper [47] demonstrates this on a husky-vs-wolf classifier whose accuracy is high but the explanation shows that it distinguishes by reading snow in the background, instead of animal features. [50] apply Layer-wise Relevance Propagation to a horse classifier and find that it reads copyright watermarks instead of horse features. [51] report that COVID classifiers trained on chest radiographs focus on body position, scanner artifacts, and hospital markers instead of actual disease features. The explanation, in all of these cases, uncovers a defect that validation in standard training processes would not detect.

3.6 Decision Mining

The concept of decision mining is an extension of process mining, which attempts to enhance process models by extracting rules for decisions and adding them to the mined process map. This approach shares the use of mining, often intrinsically interpretable, decision trees to extract the logic at the gateways. Thus it is included here as relevant prior research.

Process models contain decision points, also called gateways, where the subsequent path of a case depends on data attributes. A loan application process, for instance, might route differently depending on whether the requested amount exceeds a certain threshold. Classical process mining techniques can mine and visualize the gateway structure but leave the underlying decision logic implicit. [52] introduced decision mining to recover that logic. Starting from a mined process model, they identify gateways and determine which branch each historical case followed, then formulate the decision at each gateway as a classification task: the features are the data attributes available up to that point in the case, and the target class is the branch taken. They use C4.5 decision trees, since these produce an interpretable rule set that maps onto gateway conditions and handles mixed attribute types and missing values. [52] assumed that the event log conforms perfectly to the process model. [53] remove this assumption by first computing alignments that map each log trace to a valid replay of the process by the process model, which fills in missing events, and adjusts non-conforming behavior typical of real-world logs to match a valid process trace. Decision points are still treated as classification problems solved with standard machine learning techniques, but the discovered decision logic, referred to as guards, are part of the Data Petri Net itself, instead of a separate set of decision trees alongside the model.

This thesis adopts the same principle of building decision trees at process gateways from historical event log data as [52]. Since in PPM, we are not constrained to a fixed mined process model and instead can leverage more noisy traces, we acknowledge the alignment-based approach proposed by [53], but do not implement it. The resulting trees, however, are not used to describe the process decision logic. They serve as a ground truth for gateway routing behavior, against which surrogate trees that approximate the predictive model’s behavior are reconciled.

3.7 Summary and Research Gap

The literature review shows that PPM-specific explainability methods are scarce and when explainability methods are applied to PPM, authors typically apply techniques from other fields such as computer vision and adapt them to the model architecture. Making use of process model artefacts, on the other hand, for example to enhance visualizations appears to be underexplored, and addressed so far only by [33]. In search for a possible bridge, we reviewed the core literature in decision mining which provides a tool to represent process behavior by recovering branching logic at gateways. The review shows that the two disciplines of decision mining and explainability for PPM have so far been only pursued separately, marking an opportunity to discover synergies. And lastly, by looking into the use cases that explainability is developed for we noticed that the diagnostic use of explanations has only been applied to a limited extent to the PPM field, while being common in AI research at large. Further, the PPM literature shows that explainability research in the field has so far focused mostly on the tasks of next-event prediction and outcome prediction, while suffix prediction is largely disregarded. In the remainder of this thesis we will address these gaps by using classical explainability techniques for PPM and concepts from the decision mining and process mining fields to create two new explainability methods (gateway surrogate models and process prediction maps), all while focusing on the diagnostic use case and the developer as target audience.

4 Datasets

In this thesis we use four publicly available event logs that differ in scale, complexity, and domain. The range is deliberate and helps with validating the findings across different types of datasets. All four logs are hosted on the 4TU Centre for Research Data and are commonly used as benchmarks in predictive process monitoring.

Dataset	Helpdesk	BPIC-17	Sepsis	DomesticDeclarations
Cases (total)	3,893	26,783	892	8,925
Events (total)	37,624	1,149,864	16,941	92,585
Activities	14	26	16	8
Cat. Features	12	9	26	3
Num. Features	4	9	8	5
Trace Length (mean $\pm$ std)	9.7 $\pm$ 1.2	42.9 $\pm$ 16.0	19.0 $\pm$ 7.8	10.4 $\pm$ 1.5

4.1 Helpdesk

The Helpdesk dataset¹ records the ticket resolution process of an Italian software company’s IT support system. Each case is the lifecycle of a single support ticket, from creation to closure. The process is short and the feature set shows only a limited amount of details of the tickets. Most tickets follow one of a handful of common paths, with a long tail of rare variants.

For training and evaluation, the dataset is split case-wise into 65% training (2,977 cases), 15% validation (687 cases), and 20% test (916 cases) using a fixed random seed.

4.2 BPIC-17

The BPI Challenge 2017 dataset² comes from a Dutch financial institution and records a personal loan application process. Each case describes an application from submission to its final acceptance or rejection. Compared to Helpdesk, the dataset is much larger: more cases, longer traces, more activities, and a more informative feature set that includes domain-specific financial attributes such as the requested loan amount.

The dataset is split into 20,482 training cases (65%), 4,726 validation cases (15%), and 6,301 test cases (20%) using the same splitting strategy as Helpdesk.

¹<https://doi.org/10.4121/uuid:0c60edf1-6f83-4e75-9367-4c63b3e9d5bb>

²<https://doi.org/10.4121/uuid:5f3067df-f10b-45da-b98b-86ae4c7a310b>

4.3 Sepsis

The Sepsis event log³ records the trajectories of patients with suspected sepsis through a Dutch hospital. Each case follows a patient from emergency-room registration through triage and laboratory testing to either admission or release. Sepsis has the fewest cases of the four logs but the most elaborate and informative categorical feature space, including discretised laboratory measurements.

4.4 Domestic Declarations

The Domestic Declarations event log⁴ is part of the BPI Challenge 2020 collection and records the reimbursement of domestic travel claims at a Dutch university. Each declaration moves from submission through one or more approval stages (administration, supervisor, budget owner, pre-approver) to payment, or back to the employee for revision.

5 Models

5.1 Uncertainty-aware Encoder-Decoder LSTM

The primary model analyzed is the Uncertainty-aware Encoder-Decoder LSTM (U-ED-LSTM) proposed by [21]. It has two core properties: first, an Encoder-Decoder architecture for suffix prediction and second, an explicit treatment of predictive uncertainty that enables the model to return a distribution over suffixes or a single most-likely suffix sequence.

The model’s architecture receives a prefix of events $p_{\leq k} = \langle e_1, \dots, e_k \rangle$, which is first consumed by the encoder. The encoder returns the hidden and cell state (h_k, c_k) that together summarize the case so far. To predict the next event, the decoder is initialized with these latent states and the last prefix event e_k (also referred to as the SOS token); it produces the event prediction $\hat{e}_{k+1}$ and an updated latent state (h_{k+1}, c_{k+1}) . From there the decoder operates autoregressively, consuming its own previous output and latent state at each step until the full suffix has been generated. It is important to note that the architecture allows the decoder’s output feature set to be smaller than the encoder’s input feature set. As a consequence, the SOS token also carries only the reduced output feature set, even though at the initial decoder step the full prefix feature set would in principle be available. The output feature set is not the only part that can be flexibly adjusted in this model.

³<https://doi.org/10.4121/uuid:915d2bfb-7e84-49ad-a286-dc35f063a460>

⁴<https://doi.org/10.4121/uuid:3f422315-ed9d-4882-891f-e180b5b4feb5>

We ran the model with the configurations shown in Table 2, largely mirroring the authors’ selected settings.

For inference the model provides two sampling options. One is the most-likely suffix, where the model simply selects the most-likely successor, iteratively building the suffix in that way. The second option is to receive a distribution by means of a set of N suffixes sampled using Monte Carlo dropout of 0.1.

Configurations

In this thesis we mostly work with the most-likely prediction, as the explainability methods applied in later Chapters operate on a single deterministic predicted suffix per prefix. In the evaluation, however, we also use the distribution to assess the model’s performance. In that case we sample $N = 100$ suffixes, except for BPIC-17 where we use a sample size of $N = 25$ due to resource constraints.

Note that for the BPIC-17 and Helpdesk datasets we used the MC dropout rate of 0.0 instead of 0.1 as used by the original authors. This deviation was identified too late for a full retrain to be feasible within the scope of this thesis.

Table 2
Configurations for U-ED-LSTM

Hyperparameter	BPIC-17	Helpdesk	Sepsis	Dom. Decl.
Layers per LSTM	4	4	4	2
Output features	Activity, Resource, Time	Activity, Resource, Time	Activity, Org-group, Time	Activity, Resource, Time
Hidden size	128	128	128	64
Learning rate	1×10^{-6}	1×10^{-5}	1×10^{-5}	1×10^{-4}
Batch size	256	128	128	128
Epochs	100	100	100	200
MC dropout	0.0	0.0	0.1	0.1
Weight decay / regularization	1×10^{-4}	1×10^{-4}	1×10^{-4}	1×10^{-4}

5.2 Camargo et al.’s LSTM

As a comparable approach in the suffix prediction setting we will include the model proposed by [19].

The model is an LSTM with two stacked layers. It receives the activity, role and relative time of each event in the prefix as three input streams and predicts the same features it received as input. The two categorical attributes pass to a non-trainable embedding layer. The embedding size is fixed

at the fourth root of the number of categories. The relative time is scaled to a number between one and zero. The first LSTM layer returns a full hidden state sequence and the second returns only the final hidden state followed by a fully connected layer and a softmax output for the categorical features and scalar output for the continuous feature. There are three architectural variants that differ in how the streams share information. The specialized variant gives each stream its own pair of stacked LSTM layers. The shared categorical variant concatenates only the categorical features, activity and role, embeddings and routes them through a shared first LSTM layer before splitting into three specialised second-layer plus dense heads; the time stream keeps its own first layer. The concatenated (full-shared) variant concatenates all three streams and shares the entire first LSTM layer. The role is derived from the resource feature using the activity-profile algorithm from [54]. It creates activity-frequency vectors that represent the activities a resource performs and how often, and clusters resources that do similar activities into roles. The model is trained using fixed-size n-grams. The chosen n-gram size is typically smaller than the average trace length, except for logs with very short traces where the n-gram can fit the whole trace. At inference, the next category is sampled from the predicted distribution rather than choosing by arg-max.

Configurations

For each dataset we use the configuration [19] found best in their evaluation. For datasets that they did not use, we went with those most comparable: Helpdesk was used itself, Sepsis follows BPI 2013, BPIC-17 follows BPI 2012, and Domestic Declarations follows Helpdesk. The resulting configurations are summarised in Table 3. We deviated in two ways. BPIC-17 skips role clustering and passes the raw resource identifier directly. With 174k+ prefix windows and 145 distinct resources, the embedding layer has roughly 1,200 events per resource to learn from, which should be enough to obtain discriminative representations without the clustering. Keeping the raw resource also keeps the input comparable with the U-ED-LSTM baseline and the resource label retains more information for explainability experiments. Sepsis and Domestic Declarations also skip clustering, as they already provide a role-equivalent attribute (org:group and Role column respectively). All four configurations share Camargo’s optimization setup.

Table 3
Configurations for Camargo et al.’s LSTM

Configuration	BPIC-17	Helpdesk	Sepsis	Dom. Decl.
Scaling	max	max	log-normalisation	max
N-gram	15	5	5	5
Architecture	concatenated	shared categorical	shared categorical	shared categorical
Role source	native resource	clustering [54]	native org:group	native role

6 Explainable AI for Suffix Predictors

In Section 3, a range of explainability methods from various machine learning and deep learning domains were introduced. However, many of these methods are not plug-and-play solutions and need to be adapted depending on the model and domain in which they are applied. This can involve technical adaptations as well as new visualization approaches that account for the multi-dimensional nature of attributions in auto-regressive models, where a single prediction may involve dozens of input features across multiple sequence steps. Given the degree of adaptation required to obtain meaningful explanations for the process prediction domain, the following sections walk through the process of applying each method in detail. The methods are clustered in three groups: attribution scores, process model-based visualization and gateway analysis.

The corresponding implementation is on GitHub.⁵

6.1 Attribution Scores

The question of *What input features ultimately led to the prediction?* has been at the center of explainable AI research for a long time. Many methods as introduced in 3.3.1 have been developed for different models and domains. In this thesis we will adapt two approaches to the models at hand, one Gradient-based method and one Perturbation-based method in order to get a good understanding of what attribution scores can add to explaining the model’s behavior in general as well as how the results differ between Gradient-based and Perturbation-based methods and what those differences reveal about the trustworthiness of the explanations.

The gradient-based attribution method we use is Integrated Gradients [41] as it satisfies two key axioms: On the one hand the *sensitivity axiom*, which ensures that features that change that can

⁵https://github.com/philippeichhorn20/XAI-Probabilistic_Suffix_Prediction_U-ED-LSTM_pub

change the prediction receive a non-zero attribution. On the other hand the *implementation invariance* which describes that two equivalent networks produce identical attributions. Those properties set the Integrated Gradient apart from other methods such as the Vanilla Gradients or DeepLIFT.

The perturbation-based attribution method we selected is SHAP (SHapley Additive exPlanations) [37] due to its widespread usage in XAI research and its strong theoretical foundation in cooperative game theory.

Integrated Gradients

Integrated Gradients represent one core technique that helps explain deep learning behavior. As introduced in Section 3, Integrated Gradients provides an attribution score for each input feature of a deep learning model that represents the degree to which the prediction can be attributed to that feature. This can be leveraged on a local level, to find out what the model relied on when making a particular prediction, and on a global level by averaging the attribution scores over a larger dataset. The latter delivers a quick but meaningful overview of the features which have high relevance to the model in general, while the former can help analyze and reconstruct why a single case prediction was made.

Adaptations. One of the main challenges when using Integrated Gradients for predictive process monitoring is that it provides a large set of data points. Integrated Gradients provides an attribution score for each of the input features for each step of the auto-regressive prediction chain and for each output feature. This means that sensible aggregation, ablation and visualization are an important part of making use of Integrated Gradients.

$$\begin{aligned}
 \text{Attribution scores} &= \text{Suffix steps} \times \text{Output features} \\
 &\quad \times [\text{if categorical Output values}] \\
 &\quad \times \text{Prefix steps} \times \text{Input features}
 \end{aligned} \tag{6.1}$$

In this section we will introduce said operations used to reduce the complexity of the Integrated Gradients data points through ablation and aggregation.

Model Independent Adaptions The baseline we chose for the Integrated Gradients is the zero baseline, which sets the LSTM’s input to a zero vector in embedding space. This differs from the padding-baseline in that the zero-tensor is fed directly into the LSTM, while the padding-baseline, because the model uses zero-padding, passes the zero baseline through the embedding layer(s). For the choice of the baseline we ran a test of three baselines (zero, padding, and uniform) on the Helpdesk aggregated attribution scores to get an understanding of how they behave given the preprocessing procedure and LSTM architecture. The result showed a high Spearman rank correlation between the zero and uniform baseline ($\rho = 0.94$) and a lower Spearman rank correlation between those and the padding baseline (zero vs padding: $\rho = 0.63$; padding vs uniform: $\rho = 0.75$). We decided to use the zero baseline over the uniform baseline as the results were very similar and the zero baseline is a semantically clean choice independent of dataset distributions. And we decided against the padding baseline to allow for non-zero attributions at padded prefix positions, so that the model’s use of prefix length could be captured by attributing to those padded slots. We set the attribution steps to 50 for single case attributions and 30 for aggregated attributions. We based this on a test run of 50 Helpdesk test cases, which consistently showed a convergence delta of $<1\%$ using 30 steps.

Next Event Reduction Instead of predicting the full suffix and looking at Integrated Gradients from each step of the suffix, most of the analysis uses the simplified next event case. Next event prediction is, next to outcome prediction, the most studied task in PPM. Additionally, in autoregressive suffix prediction models, the subsequent steps are based on potentially incorrect earlier predictions, making the attributions less reliable. Focusing on the first step therefore provides a cleaner and easier to interpret signal.

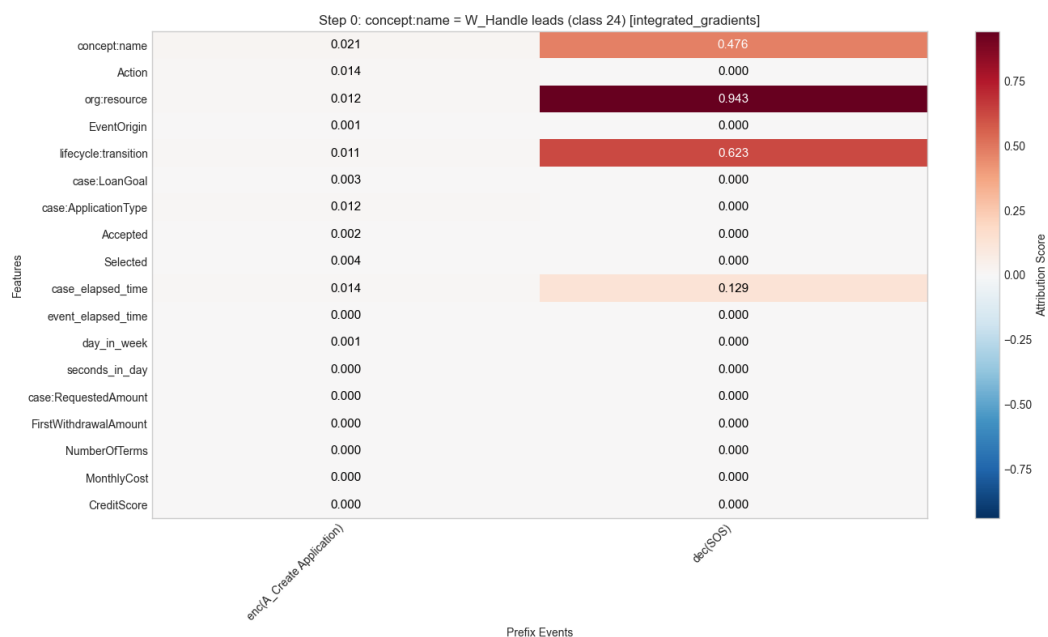
Predicted Class Reduction The second measure is to focus on the predicted class when computing the attributions for categorical output features. Rather than calculating Integrated Gradients with respect to all possible output values, we compute them only for the value the model predicts. This matches the idea of explaining why the model decided to predict a certain value. However, it comes at the cost of omitting the related explanation of why the model predicted the other values to a lesser extent.

Single Case Observation - Local. The most straightforward use case of Integrated Gradients is the single-case analysis. By calculating the IG, the user obtains an attribution score for

each of the input features, which explains to what degree each feature contributed to the model's prediction. Positive values indicate a contribution in favor of the prediction, while negative values indicate the opposite. Given the high dimensionality of the attribution tensor introduced above, the choice of visualization largely determines which aspects of the explanation become accessible. In the following, we introduce two complementary heatmap visualizations: one that focuses on a single prediction step but shows the full feature set, and one that spans the entire suffix but is restricted to the most relevant features.

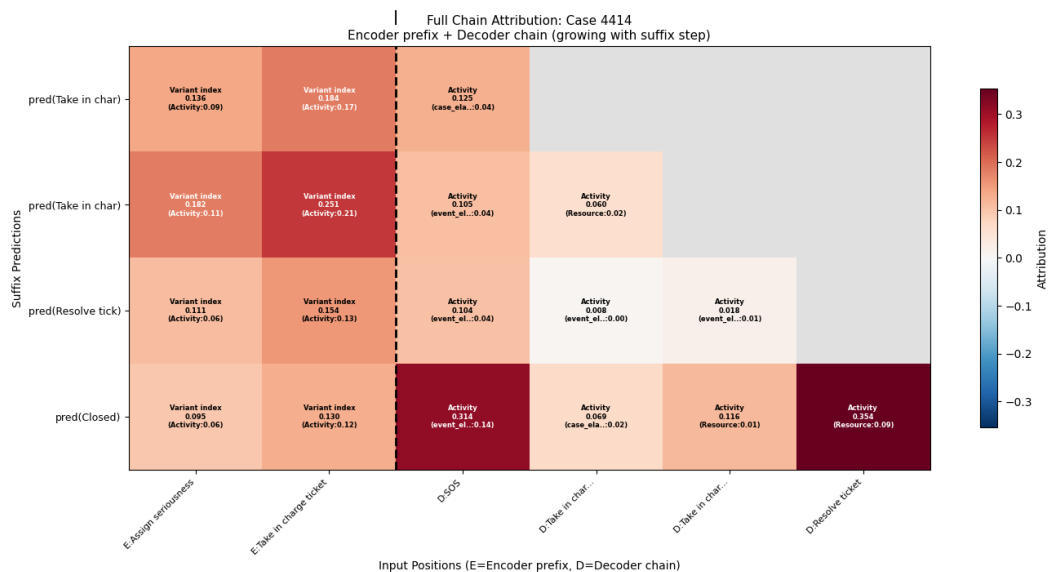
Next-Event Heatmap The first visualization focuses on a single prediction step and shows the attribution score for every input feature. An example is given in Figure 2, which depicts the first prediction step of a suffix with length one. The full set of input features is listed along the y-axis, while the prefix steps are arranged along the x-axis. For the encoder-decoder model at hand, the encoder and decoder input have to be evaluated separately because a subset of features is only inserted into the encoder and not the decoder which is why even though the example prediction only has a prefix of length one, two prefix events are shown on the x-axis. For those features that are only entered into the encoder, the attribution score on the decoder side are zero. This visualization helps the user understand what features the model attributes its prediction to and which it does not. The score is also directional, meaning that features with positive scores influenced in favor of the prediction and features with negative scores opposed the prediction.

Full Suffix Heatmap While the next-event heatmap gives a complete view of the feature space for a single prediction step, it does not show how attributions evolve across the subsequent steps of an auto-regressive suffix prediction. The full suffix heatmap, shown in Figure 3, trades feature breadth for temporal depth: the prediction steps of the suffix are arranged along the Y-axis and the steps in the prefix along the X-axis. Because the encoder-decoder model feeds each predicted step back into itself for subsequent predictions, the prefix grows with each suffix step, which produces the characteristic staircase structure of the heatmap. To understand how the model arrived at a particular prediction, the reader locates the corresponding row on the Y-axis, for example, the prediction "Take in charge" and reads the cells to the right as its inputs. For each input step, the heatmap reports the feature with the highest attribution score, with the second highest shown in parentheses. To keep the figure legible, only the top two features per cell are displayed rather than the full feature set.

Figure 2*Next-event Attribution Heatmap for a BPIC17 Prefix*

Adaptations for the Encoder-Decoder LSTM The input steps are split into encoder and decoder inputs, which is a model-dependent distinction. In the U-ED-LSTM model, the encoder receives the full prefix as input, while the decoder receives the hidden state produced by the encoder together with the so-called SOS token, a separate encoding of the last step of the input sequence (in this case, "Take in charge"). For the next prediction step, again "Take in charge", the SOS input would be the previous "Take in charge" prediction. This split is necessary because the SOS encoding differs from the rest of the input sequence and only contains a reduced set of input features compared to the encoder input. Looking at the full suffix heatmap, it becomes clear that the prediction is primarily driven by the variant index and, to a lesser extent, by the activity. This matches the expected behavior: the variant index is assigned after process completion and encodes entire activity sequence patterns. The model uses the variant index to identify which sequence the case belongs to and the activity to locate its current position within that sequence. While the variant index may not be available in a production setting and may need to be excluded from the feature set, its dominance in the attributions provides a clear view of how the model's learned patterns are structured. Disregarding the variant index, the final prediction "Closed" reveals a different learned behavior. More than 95 percent of the sequences end with the two activities "Resolve ticket" and then "Closed". Accordingly, when producing the "Closed" prediction, the model heavily attributes its decision to the previous prediction's activity (D-SOS and D-Resolve ticket).

Figure 3
Full Suffix Attribution Heatmap for a Helpdesk Prefix

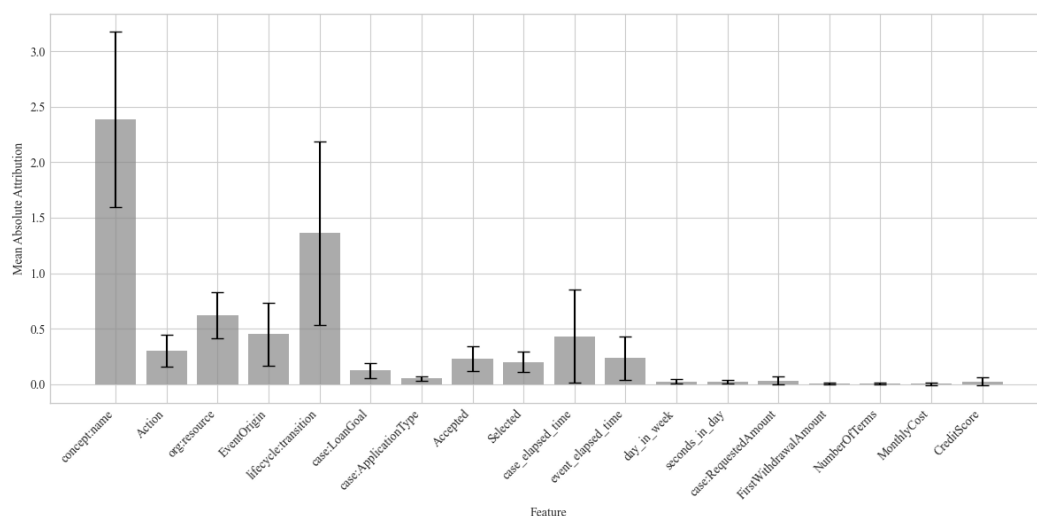


As shown in the examples above Integrated Gradients can help the user of a PPM model reconcile the decision behavior of a model with its expected behavior. When the user knows which input feature is most relevant for a certain decision, checking if the model also attributes the prediction to that feature, provides greater trust in that model and uncovers issues if it does not. This is especially useful when trying to analyze why the model does not predict correctly.

SHAP

For Shapley values (SHAP) we use the permutation-based Shapley variant introduced in Section 3.3.1, which has previously been used in PPM on tabular features. Our setting is autoregressive suffix prediction on raw event sequences, where the standard sampler does not transfer cleanly: every position would otherwise become its own Shapley player, inflating the number of evaluations and the granularity of the explanation. We adapt the sampler by separating case-level from event-level features. We distinguish between event-level and case-level input features. Case-level features are constant across positions and are treated as a single Shapely player which can only be switched across all positions simultaneously. Event-level features remain a single player per timestep. The baseline is zero, matching the choice in Integrated Gradients, and we use $n=30$ permutation samples.

Figure 4
Mean Absolute Attribution Scores of U-ED-LSTM on BPIC-17



Data Set Aggregation - Global

To get a more general overview of which features the model relies on, we aggregate the attributions over the full dataset and break them down by input feature (Figure 4). While this loses the per-case nuance, it provides a quick overview of which features the model bases its predictions on. The attribution score we use for the aggregation is Integrated Gradients, because computing SHAP across the full test set is infeasible at this scale. We aggregate the absolute Integrated Gradient values, so that opposing signs across cases do not cancel out and the result reflects the magnitude of the model’s reliance on each feature. The caps on top of the bar charts visualize the variance by showing the range of the mean plus/minus the standard deviation.

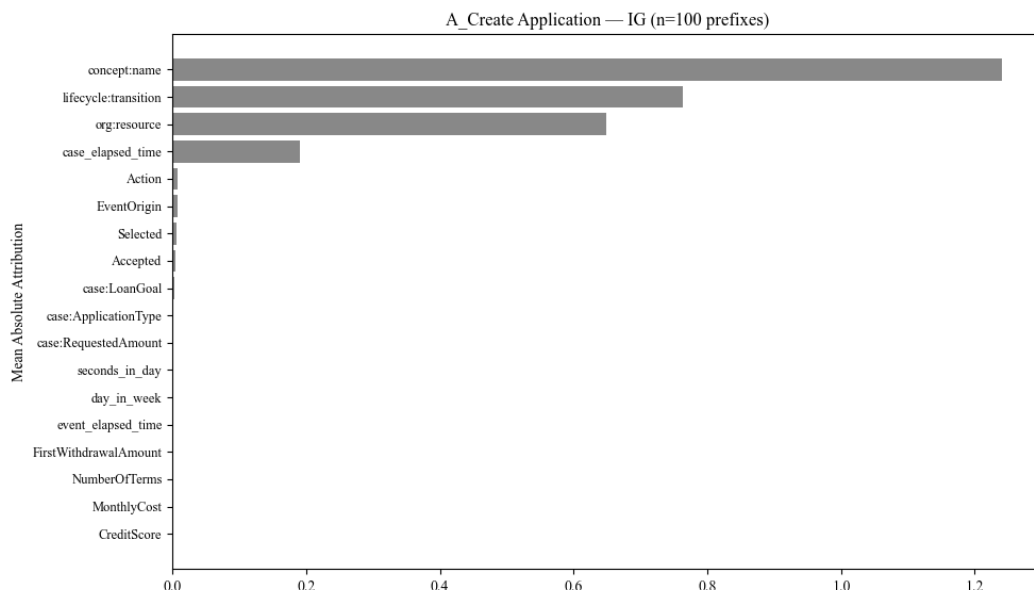
The same aggregation procedure can also be restricted to prefixes that end in a given activity, typically a gateway within the process (Figure ??), so the reader sees what the model relies on when predicting the successor of that specific activity. This bridges the full-dataset aggregation and the gateway analysis introduced in Section 6.3 as it localizes the attribution to a single decision point while still averaging over enough cases to reduce the noise of single-case attribution maps.

6.2 Process Model-Based Visualization

As mentioned earlier a core part of explainability is helping the user to understand why the model predicts a certain event and by extension uncover patterns of the model’s predictive behavior. By leveraging the existing methods of the process domain such as process maps we introduce the

Figure 5

Mean Aggregate Integrated Gradient at Gateway "A_Create Application" of U-ED-LSTM for BPIC-17



process prediction map (Figure 6), which is a process map that visualizes the PPM model's prediction accuracy along the control flow.

The motivation for using a directly-follows graph (DFG) over an inductive miner combined with a Petri net (or any other higher-level process representation) is that the DFG does not infer any process logic which may or may not match the underlying behavior. Instead, the DFG only visualizes the actual behavior observed in the test cases. This one-to-one correspondence is exactly what we want for a prediction overlay: each edge in the DFG corresponds to one or more next-event predictions made by the model, so every visible transition in the map is a decision the model was actually asked to make. Using an inferred process model would instead introduce edges that do not appear in the data and hide edges that do, which would blur the link between what the map shows and what the model was evaluated on.

1. Mine a directly-follows graph $G = (V, E)$ from the test cases in the event log.
2. For every case in the event log and every non-empty prefix of length $k \geq 1$, query the prediction model and record the predicted next activity.
3. Aggregate the predictions into transition-level statistics of the true transition between two nodes (u, v) . For each such pair count how many prefixes ending in u with true successor v were correctly predicted by the model. Call this the count of correct predictions $c_{u,v}$ and

denote the count of occurrences of the transition in the test cases as $n_{u,v}$. Further, define the accuracy of the transition as $acc_{u,v} = c_{u,v}/n_{u,v}$. Finally define the accuracy of a node as the aggregated accuracy over all outgoing transitions $acc_u = \sum_v c_{u,v} / \sum_v n_{u,v}$.

4. Add the accuracies as fraction of form $c_{u,v}/n_{u,v}$ and c_u/n_u to all edges and nodes as labels, to keep the information of the accuracy but also how common the transition or node is.
5. Color the edges and nodes with a continuous gradient (red to yellow and yellow to green) where high accuracies are green and low accuracies are red.
6. Add a "Start" node that has flows to all the activities that start a sequence and a label with how often they are starting activities.

This results in a map that provides a nuanced yet big picture overview of the model's performance. It lets the user instantly identify decision gateways where the model underperforms. Three interesting patterns to look out for are:

1. **Green node with multiple outgoing flows.** The model has successfully generalized and can correctly predict what the next event is not just based on the current event. An example is in the BPIC17 process prediction map the W_Call node that follows the offer events, where the model correctly predicts if the next event is A_Complete or a self loop.
2. **Red node.** The model does not correctly predict the successor events of that event. The model thus has not learned what role that event plays in the sequence flow.
3. **Event with multiple outgoing flows where some are green and some are red.** In these cases, the green flows are typically much more frequent than the red ones. This suggests that the model has overfitted to the most common sequence pattern and, in order to minimize the overall error term, disregarded the less common logic and overfitted to the majority flow.

The process prediction map visualization does have some natural limitations. Firstly, since the model only uses next event predictions and not full suffixes it disregards bad performance on longer suffix predictions. However, removing this uncertainty makes it clearer at highlighting next-event shortcomings, which are typically the first indication of a model limitation and the root cause of cascading errors in longer suffixes. Also it only focuses on the activity feature, meaning shortcomings in the time prediction or other event attributes are not captured by the map.

Figure 6
Process Prediction Map of U-ED-LSTM for Helpdesk

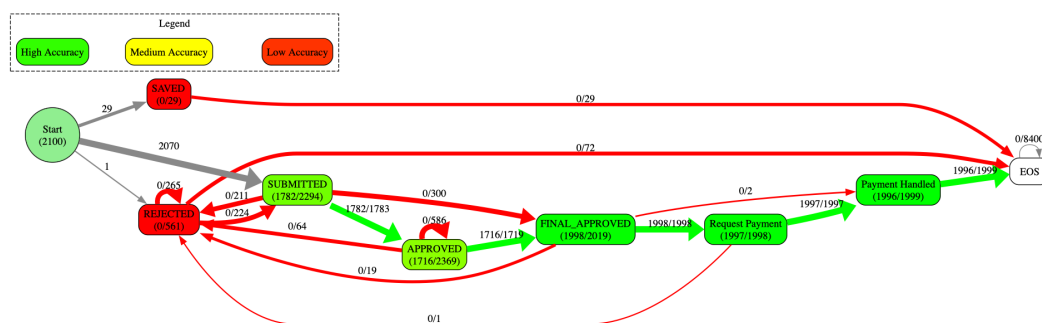
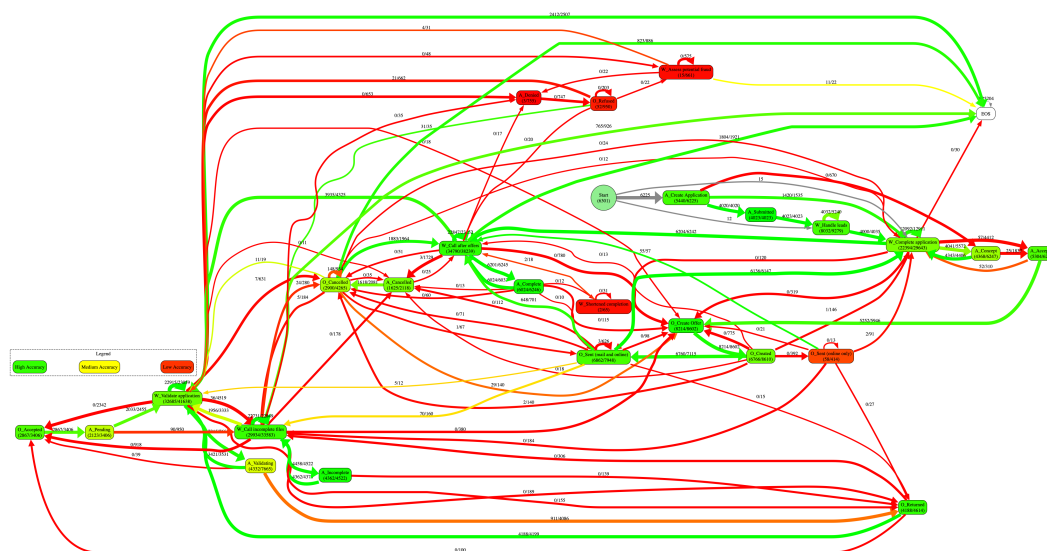


Figure 7
Process Prediction Map of U-ED-LSTM for BPIC-17



The process prediction map grows with the number of activities in the dataset. Datasets with a larger activity set, such as BPIC17 (Figure 14, therefore produce a map that appears rather cluttered at first. Despite the clutter, the coloring gives the reader a rough sense of overall performance at first glance and quickly directs attention to the model's problematic areas, usually marked by red arrows or nodes.

6.3 Gateway Analysis

Gateways represent the branching logic that determines which event follows when multiple continuations are possible. While in process models these gateways are formally modeled using XOR and AND nodes, in PPM models these gateways remain implicit and only manifest in different predictions for different prefixes. Extracting the logic behind these gateways is a core challenge that

the PPM model has to learn in order to make correct predictions. Analyzing whether the model has indeed learned the correct gateway logic can help explain the model's decisions. To understand what decision logic the model has learned, we use surrogate models which are only trained on the model's predictions at a singular gateway and which are inherently interpretable and can thus be analyzed by a human.

Surrogate Trees at Gateways

Surrogate models have been used to model neural nets at specific decision points since the 90s [55]. The most common types of surrogate models are decision trees and simple linear models. In our case, we go with decision trees as they have been used to model decision gateway behavior in the decision mining domain. This gives us two advantages: first, a method that has proven suitable for representing gateway decision logic, and secondly, it will later in Section 6.3.2 give us the chance to compare our surrogate trees of the model with the actual trees trained with the ground truth to then check if they are the same.

To construct the per-gateway surrogate models, we first identify the decision points in the process at which branching occurs. A gateway is defined as an activity that has more than one distinct successor activity observed in the event log; we additionally require each gateway to have a minimum number of prefixes and at least two branches with sufficient support. For each such gateway, the prediction target is the immediately following activity: given a prefix ending at the gateway, the model predicts which activity will be executed next. The surrogate tree is fitted on cases drawn from the test set, i.e. data not seen during model training. This test set is partitioned via a stratified 60/20/20 split into a fitting set used to grow the surrogate tree, a validation set used to select the tree depth (shallowest tree within a 2 percent tolerance of the best validation score), and a held-out test set for the analysis.

To attain the surrogate tree, a set of trees is built using the C4.5 algorithm, all with maximum depths between 3 and 8. Of those trees, the shallowest one is selected where the additional depth increment does not increase the fidelity of the tree by more than 2 percentage points.

Feature engineering for the surrogate trees The input of the surrogate tree is a set of features derived from the prefix that ends at the gateway activity, listed in Table 8 for the BPIC17 dataset and in Table 9 for the Helpdesk dataset. The features include the prefix length as the only purely

engineered scalar, case-level features (features that stay constant over the course of the case in at least 95 percent of the cases), and event-level features, both categorical and numerical, which are enumerated over a window of 40 positions for BPIC17 and 10 positions for Helpdesk (for example, *concept:name_pos_-1* or *org:resource_pos_-10*). The threshold of 95 percent is set permissively to tolerate occasional data quality issues such as a single inconsistent resource string within an otherwise static case, and the window sizes are chosen to reflect the differing trace-length distributions of the two event logs.

Of all the resulting surrogate models, we exclude those that have a fidelity of less than 85 percent to ensure that we only work with surrogate models that capture the model’s decision behavior to a reasonable extent.

This leaves us with one decision tree per gateway mimicking the decision behavior of the model. The derived decision logic is explicitly written on the tree, making it easy for the reader to interpret.

The main limitation of surrogate trees that the simpler surrogate tree in order to be inherently interpretable can not capture some of the complex decision logic that the deep learning model can, due its limited number of nodes and edges.

Surrogate Tree vs. Decision Tree

In this section, we introduce an approach that combines research on decision mining with explainability research to derive a new method to explain model behavior.

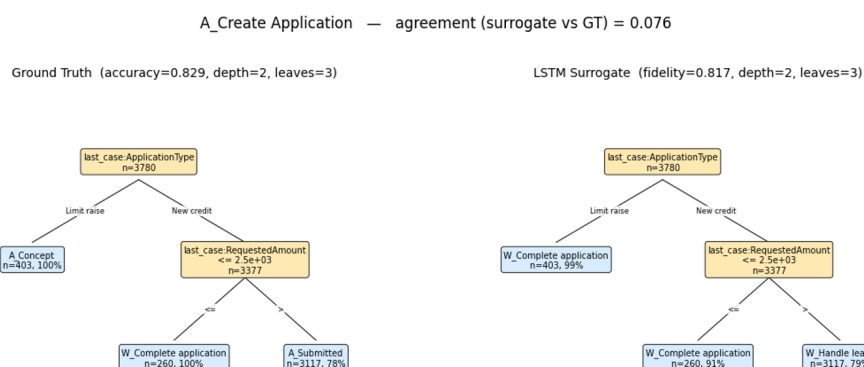
As mentioned in 3.6, modeling gateway decisions as a classification problem and training a decision tree on it has been successfully implemented for the purpose of decision mining [53], [56]. At the same time, the surrogate models at gateways produce a similar decision tree.

Combining these two approaches offers a great insight because it lets the user not just see if the model prediction differs from the expected value but if the decision making process (or at least an approximation of that) is consistent.

Since both decision trees are approximations, it is important to ensure a high fidelity of the surrogate decision trees. This ensures that we only look at gateways where the underlying decision logic can be captured by the decision tree. Thus we limit this approach to gateways where the ground truth

Figure 8

Example of a Ground Truth Decision Tree and Gateway-specific Surrogate Tree



and the model predictions can both be approximated by a high fidelity (over 85%) by their respective model. While this limits the applicability it ensures trustworthiness in the results.

These methods are applied in Chapter 7 to the two PPM models across four datasets.

7 Analysis

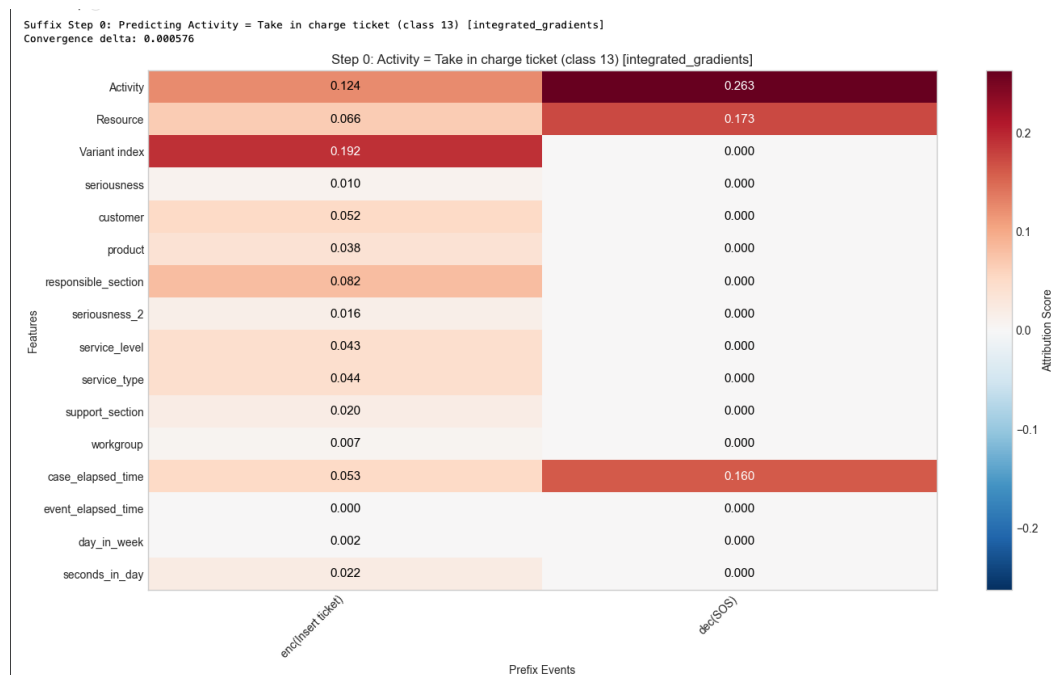
In this Chapter we use the explainability methods from Chapter 6 to analyse the model behavior and further spot areas for improvement which go beyond of what is identified during model training. Throughout the analysis we will look for recurring patterns that are then formalised in Chapter 8 as a set of Signals that identify potential for model improvement.

We conduct the analysis on 2 different model architectures as introduced in Chapter 5 which were trained on four datasets, Helpdesk, BPIC-17, Sepsis and Domestic Declarations which were introduced in Chapter 4.

7.1 Uncertainty-aware Encoder-Decoder LSTM (U-ED-LSTM)

Helpdesk

We start the analysis with a top-down approach using the process prediction map in Figure 9 which gives a global picture of the models precision patterns while also offering nuanced and gateway specific explanations. It quickly shows that the model's prediction quality varies strongly between

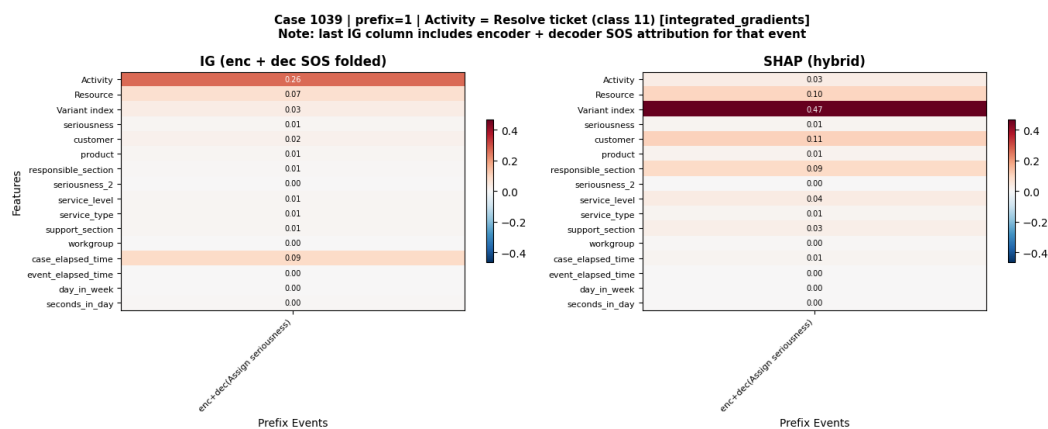
Figure 10*Next Event Integrated Gradient Score of Example Case*

Assign seriousness activity through inserting *Insert ticket* activities before *Assign seriousness*, the model may be able to learn the behavior as expected. To better understand why the model does not change its prediction when the prefix starts with *Insert ticket* instead of *Assign seriousness*, the Integrated Gradients for an example case were looked at and are shown in Figure 10. The attribution scores mostly point to the activity label and the variant index, which suggests that the model is using the expected features and not relying on unrelated ones. However, Integrated Gradients only show which features influence the prediction, not how the model actually interprets them. This means the result fits both the correct behavior and the replacement hypothesis, and does not rule out the latter. The reason for the wrong prediction therefore has to be looked for somewhere beyond feature attribution.

Another activity which is highlighted through the process prediction map is *Assign seriousness* itself, since only half of its successors are predicted correctly despite the activity being very frequent in the training data. This suggests that, in theory, the model should have enough data points to form an internal representation of what activity succeeds *Assign seriousness*. However, the model predicts *Resolve ticket* as the successor in 445 out of 995 (45%) test cases, whereas it is the true successor in only about 50 out of 995 (5%) cases. The model, therefore, performs worse than a

Figure 11

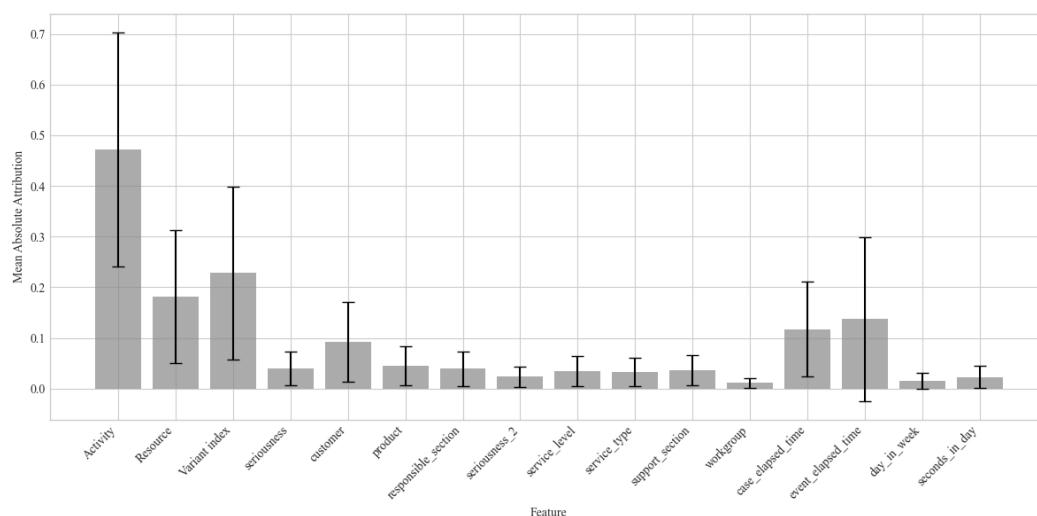
Attribution Scores (IG and SHAP) for Sample Prediction



baseline, which always predicts the majority-path successor. The majority path to *Take in charge ticket* is only predicted correctly (that is, in those cases where the true successor is indeed *Take in charge ticket*) in 507 out of 837 (61%) test predictions. A collapse to the majority path would instead yield 837 out of 995 (84%) correctly predicted successors, which would be a stark improvement over the current 538 out of 995 (54%) correct predictions. This indicates that the model has not converged to even a trivial decision rule at this decision point, despite the abundance of training examples. To investigate the source of this miscalibration, a local surrogate decision tree was trained on the prefixes ending in *Assign seriousness*, and attribution scores were computed using Integrated Gradients and SHAP. The surrogate tree could not identify any discriminating factors and collapsed to the majority prediction, which suggests that the discriminative signal lies outside the features available to the surrogate. The two attribution methods, as shown in Figure 11, disagreed in the ranking of the top-attributed input features, which casts doubt on the reliability of these methods in this setting.

Next to the process prediction map, the Integrated Gradients aggregated over the test dataset (Figure 12) can be used to get a global understanding of the model behavior and more specifically, what the driving input features of the model are. In this case, the Variant index appears as the second most attributed input feature. Since the Variant index is not part of the features which are also entered into the decoder, like the other 4 most attributed features, this further stands out. Since the Variant index is assigned to the activity sequence, i.e. two sequences with the same activities have the same variant index, it is sound that the model would highly rely on such a feature for activity prediction. At the same time, since the Variant index is assigned after the case is completed by the process

Figure 12
Aggregated Integrated Gradients of U-ED-LSTM on Helpdesk



tool and in real word process prediction task it would not be given as a feature, it may be sensible to exclude it from the training data. Alternatively it could be masked during the model training to ensure the model generalizes past the Variant index and make the model more robust for real world use cases.

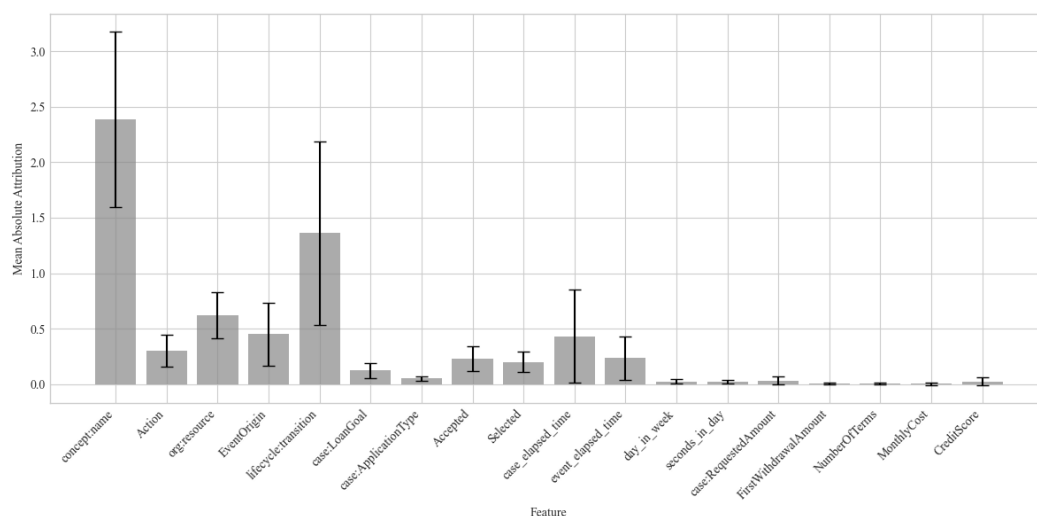
BPIC-17

We start with the aggregated Integrated Gradients over the test cases to see which input features the model relies on. As shown in Figure 13, the two features dominating are "concept:name" (the activity label) and "lifecycle:transition" (whether the event is the *start* or the *complete* of its activity) with the "org:resource" at third. The activity label is the expected dominant feature for next-event prediction. The high attribution to *lifecycle:transition* reflects the structural property of the BPIC-17 event log that many activities appear as paired start/complete events, so knowing which phase the case is currently in determines whether the next event is likely again the same activity or a transition to a new one. The relatively low attribution to case-level loan attributes such as "case:RequestedAmount" and "case:ApplicationType" suggests that the model primarily relies on event-level signals and rarely uses case-level information to discriminate between branches.

In the second step we leverage the process prediction map in Figure 14 to identify spots within the process where the model struggles and where it performs well. The process performance maps quickly shows the activities are split into green ones, where the successor is overwhelmingly

Figure 13

Aggregated Integrated Gradients of U-ED-LSTM on BPIC-17



predicted correctly and red ones, where the next activity is almost always predicted incorrectly. Those red activities are all occurring in less than 20% of the cases, making them outliers. However even for *A_Denied* the successor is not predicted correctly, even though it is the same one in 99% of the cases (*O_Refused*). A closer look at the predictions for prefixes ending in *A_Denied* shows that the model predicts *W_Validate application* as the successor in the overwhelming majority of cases (roughly 95%), and *W_Call incomplete files* in the rest. *W_Validate application* is, alongside *EOS*, one of the dataset's most-predicted activities overall, and that is what the model has fallen back on at this gateway.

The process prediction map also shows the model picking up clear decision behavior at several gateways. The clearest example sits right at the start of the process, where the starting activity *A_Create Application* branches into one of three successors: *A_Submitted*, *A_Concept*, or *W_Complete application*. Figure 15 shows the ground-truth and surrogate decision trees at this gateway. The ground-truth tree splits on *ApplicationType* and *RequestedAmount*: if the application is a Limit Raise, the next activity is *A_Concept*, if it is a new credit, the successor depends on the amount, with *W_Complete application* following requests below 3500 euros and *A_Submitted* following requests above. The surrogate tree on the right of the figure makes the same distinctions.

The model captures two of the three paths correctly. On the Limit Raise path, however, it predicts *W_Complete application* instead of *A_Concept*. *A_Concept* is the rarest of the three branches,

Figure 14
Process Prediction Map of U-ED-LSTM on BPIC-17

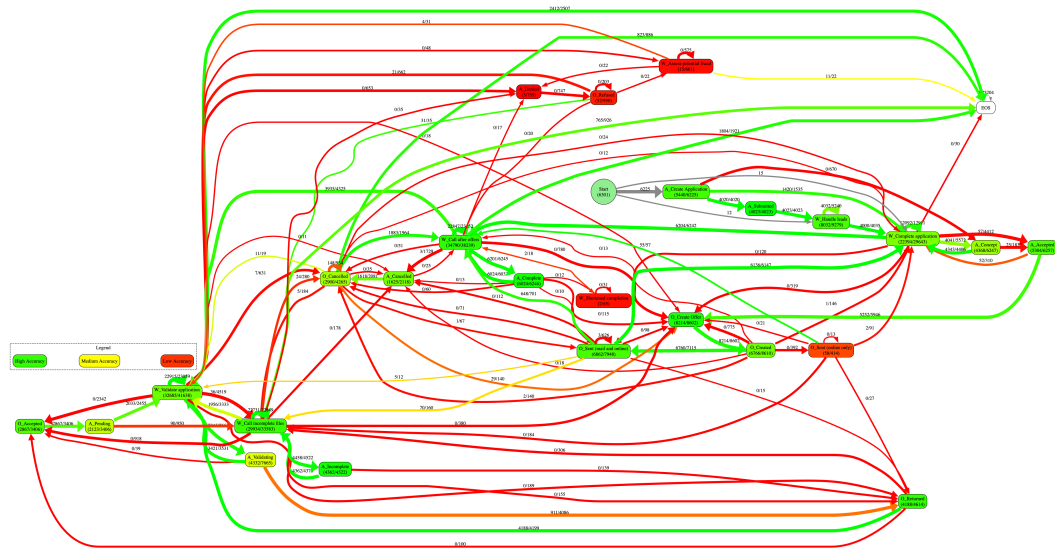


Figure 15
Decision Trees for U-ED-LSTM on BPIC-17 at Gateway "A_Create Application"

A_Create Application — agreement (surrogate vs GT) = 0.894

Ground Truth (accuracy=0.829, depth=2, leaves=3)

LSTM Surrogate (fidelity=0.830, depth=2, leaves=3)

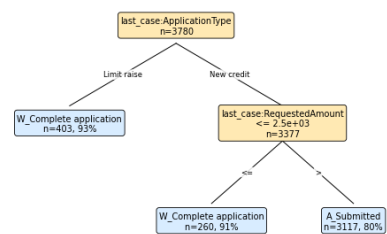
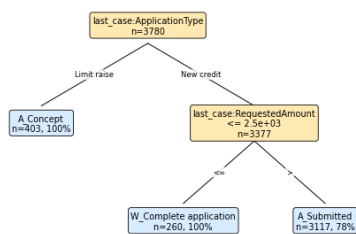
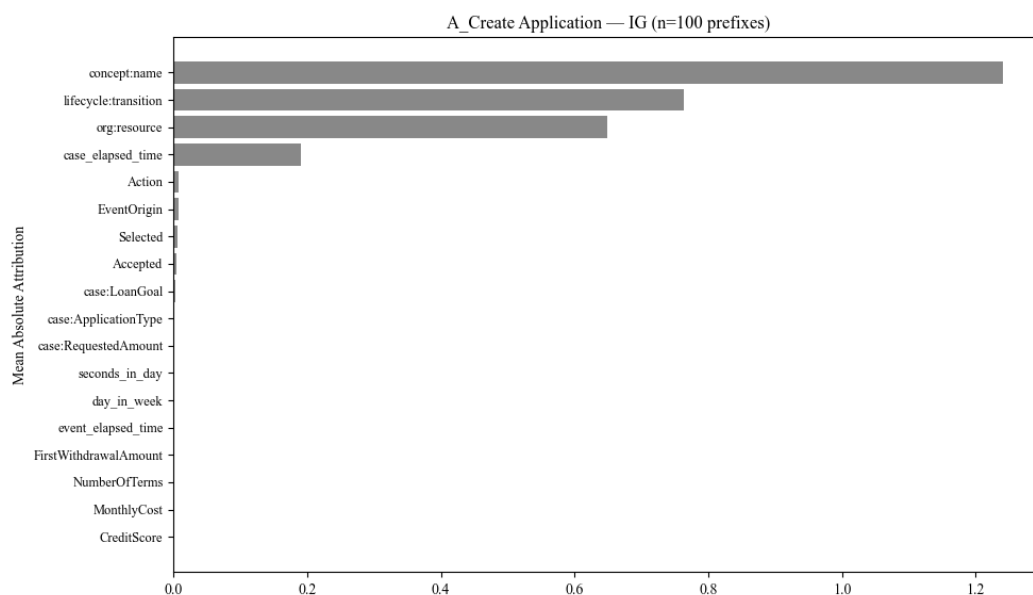


Figure 16

Mean Aggregate Integrated Gradient at Gateway "A_Create Application" of U-ED-LSTM on BPIC-17



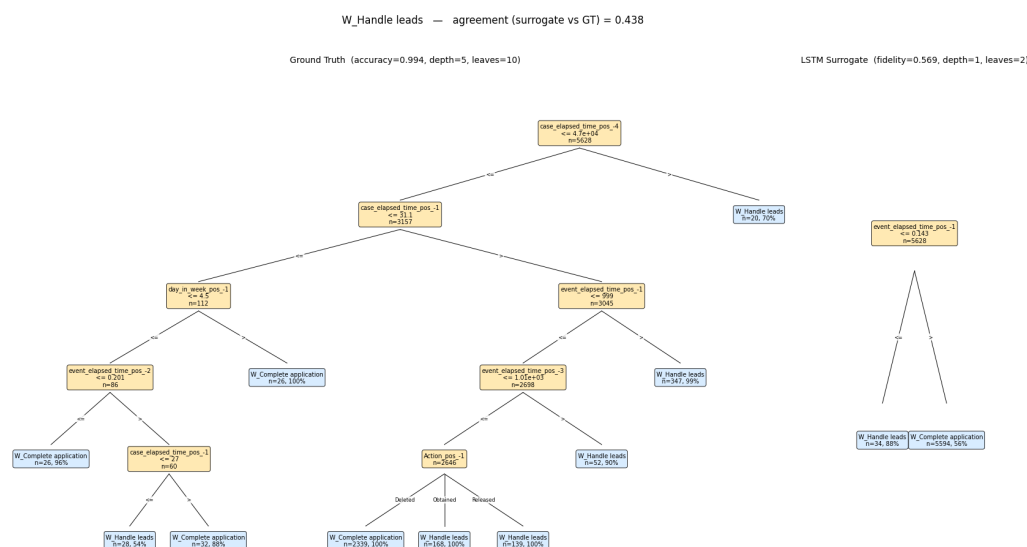
accounting for only about 10% of the 6225 cases at this gateway. It is also the one the model has failed to learn.

The attribution scores (Figure 16) at this gateway show that the model does not focus on the features the gateway actually depends on, "ApplicationType" and "RequestedAmount". The highest-attributed features are "concept:name", with 1.24 versus 0.01 for "RequestedAmount" and 0.002 for "ApplicationType". This points to a learned shortcut. A plausible candidate, given the structure of the data, is that the model uses the "org:resource" feature (this feature represents the user) as a proxy, inferring that certain employees are typically associated with requests above the threshold amount.

Another noteworthy case is the *W_Handle leads* gateway. Here a ground-truth decision tree significantly outperforms the U-ED-LSTM: the tree predicts the next activity with 99% accuracy (Figure 17), while the model reaches only 87% (8032 out of 9279). The decision the model has to make at this gateway is whether the case loops back to *W_Handle leads* or transitions to *W_Complete application*. The splitting pattern is not trivial, but the ground-truth tree captures it reliably. A surrogate fitted to the LSTM's predictions, by contrast, cannot recover any consistent rule, which suggests that the LSTM has not learned a coherent decision pattern at this gateway even though it exists.

Figure 17

Decision Trees on U-ED-LSTM for BPIC-17 at Gateway "W_Handle Leads"

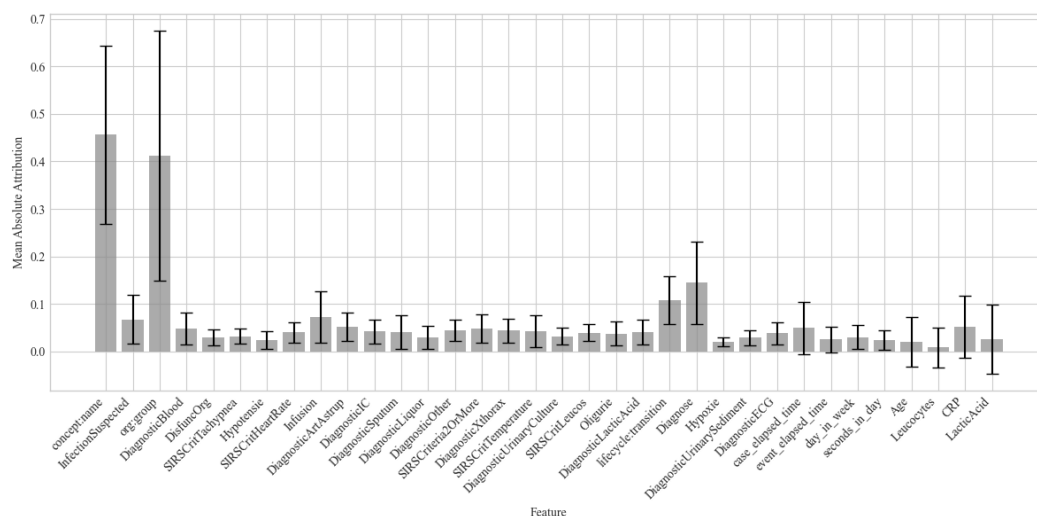


Sepsis

The aggregated Integrated Gradients over the test set, as shown in Figure 18, reveal that the model puts only very limited weight on the medical variables and instead leans mostly on the process attributes "concept:name" (0.46) and "org:group" (0.41). Among the medical variables, on the other hand, only "Diagnosis" receives some attribution (0.14). This puts in doubt whether the model has actually picked up a lot of the underlying medical logic, even though the input feature set contains a wide range of medical data points per case.

The process prediction map in Figure 19 shows that almost every node in the start-to-mid-process region has only one green outgoing flow, all pointing to CRP. The model therefore predicts *CRP* as the successor regardless of which prefix ends at the node. As a result, every *CRP*-incoming transition is captured near-perfectly (e.g. *Leucocytes* to *CRP* with 330 out of 330), while every other start-to-mid-process transition is predicted with 0% accuracy, including dominant majority flows such as *ER Registration* to *ER Triage* (0 out of 188), *ER Triage* to *ER Sepsis Triage* (0 out of 179), and *IV Liquid* to *IV Antibiotics* (0 out of 85). *Admission NC* is the one partial exception, where the model discriminates between *CRP* (31 out of 65) and *Leucocytes* (25 out of 77). In the late-process region, the single-flow gateways *Release B* to *EOS* (10 out of 10) and *Return ER* to *EOS* (48 out of 53) are predicted correctly, but the two-way *Release A* gateway falls below the majority baseline

Figure 18
Aggregated Integrated Gradients of U-ED-LSTM for Sepsis



since the model picks the minority *Return ER* branch (49 out of 53) and misses the dominant *EOS* branch (1 out of 81). In combination with the near zero attribution of medical features mentioned before, this further shows that the model is unable to capture the process flow logic in the current training setup.

Even though the results above suggest that the logic the model has learned is at most limited, the partial discrimination at *Admission NC* hints that the model has picked up some pattern. To check whether this pattern is recoverable, we fit a surrogate decision tree to mimic the model's predictions at this gateway. The tree reaches a fidelity of only 0.5 (Figure 20), meaning it agrees with the model on circa half of the cases and is therefore too unreliable to use as an explanation. As a further test, we also fit ground-truth decision trees on the available features at each gateway to see whether simple decision rules exist that the model may have missed. None of the resulting trees reaches high accuracy, and none of them produce a rule that matches the domain or process logic. Either the underlying decision logic is too complex for a simple decision tree, or the available features do not carry enough information. In both cases, the training setup has not captured it successfully.

Domestic Declarations

Regarding the model using the U-ED-LSTM, Figure 21 shows the process prediction map's ability to showcase model behavior at one glance. It appears that the model's prediction has collapsed to the majority successors for all but one activity. The one activity, namely REJECTED, remains largely

Figure 19
Process Prediction Map for U-ED-LSTM on Sepsis

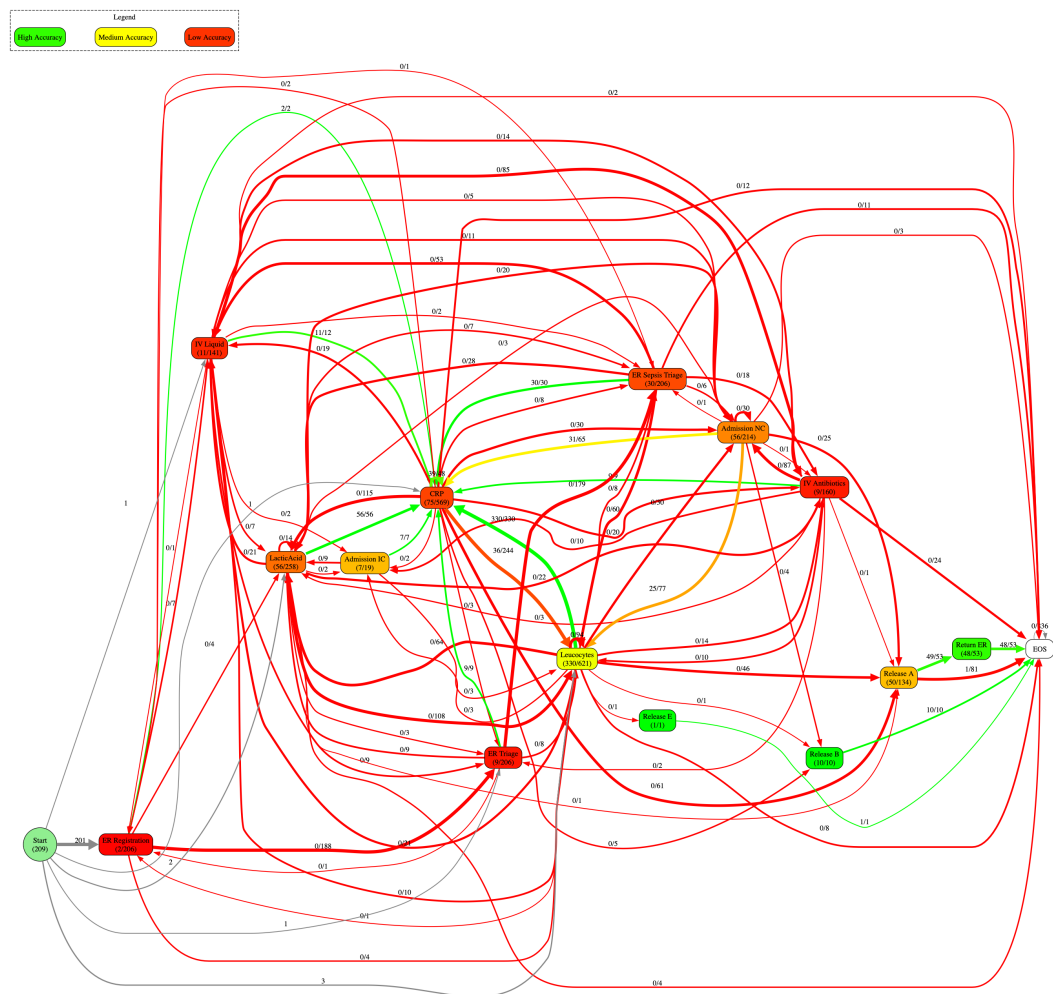


Figure 20
Decision Trees on U-ED-LSTM for Sepsis at Gateway "Admission NC"

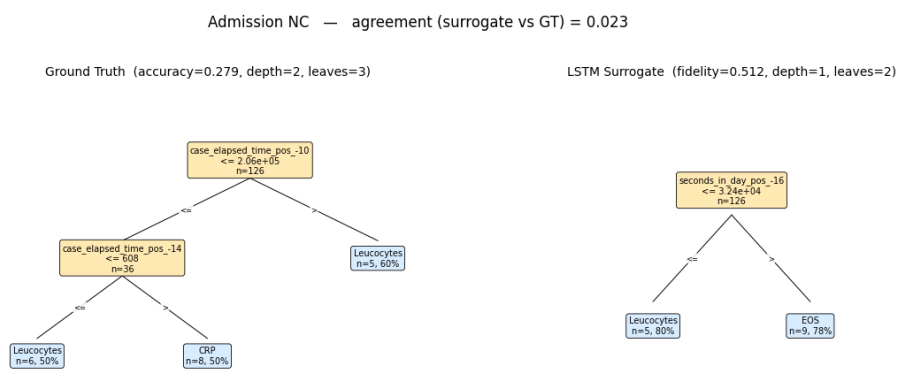
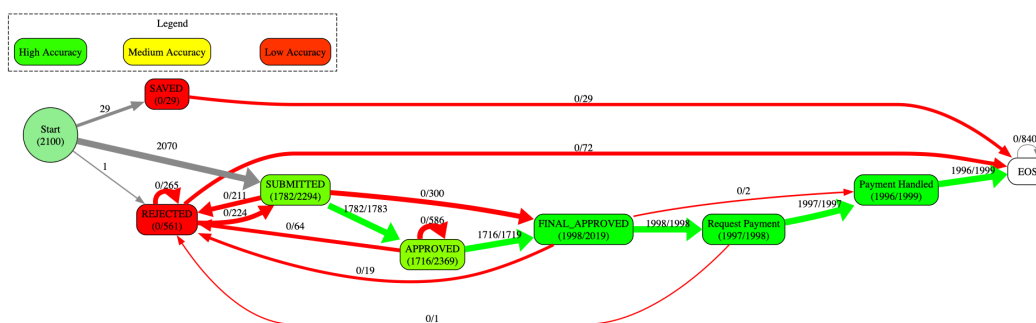


Figure 21
Process Prediction Map for U-ED-LSTM on Domestic Declarations



invisible in the model's predictions. Since the activity is common and has a clear path (almost every REJECTED activity is first repeated, then another event takes place), this shows a suboptimal training flow. Further analysis at this point is not needed as the model's learned pattern are clear.

7.2 Camargo et al.'s LSTM

Helpdesk

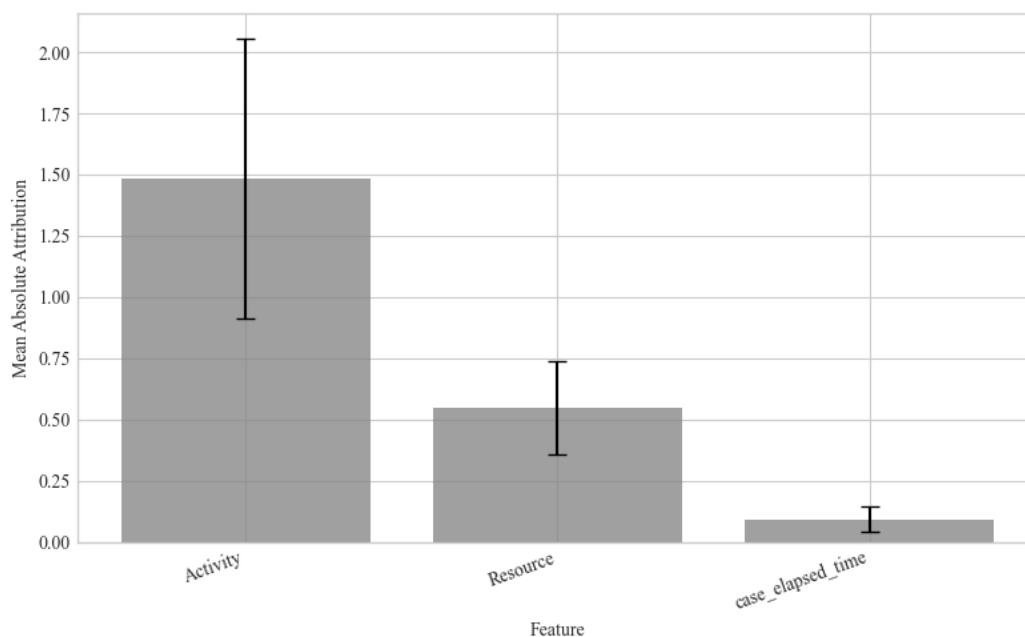
For the analysis of Camargo et al.'s LSTM on the Helpdesk dataset we begin with the aggregated Integrated Gradients over the test set, shown in Figure 22. The attribution covers the three features that the model receives as input: the activity label, the resource, and the elapsed case time. The activity dominates with a mean absolute attribution of roughly 1.5 per prefix, the resource follows at around 0.5, and the elapsed time remains below 0.1.

This is consistent with the business context: the next event in a Helpdesk ticket is largely determined by the activity sequence so far. The responsible resource provides additional discriminative signal, while the time elapsed since the start of the case carries little information. In contrast to the U-ED-LSTM analysis, no variant index is present here, so the leakage concern identified in the previous section does not apply to this model.

When analyzing the process prediction map for Camargo et al.'s LSTM in Figure 23, it first shows that the model performs much better on the early activities *Insert ticket* and *Assign seriousness* than the U-ED-LSTM does. For *Insert ticket* it appears to correctly recognize its position in the process flow, and for *Assign seriousness* Camargo et al.'s LSTM simply collapses to the majority successor, which is a more convenient default. The collapse to the majority path is, in general, a consistent pattern across almost all activities, with the exception of the outlier *Resolve SW anomaly*, whose

Figure 22

Aggregated Integrated Gradients of Camargo et al.'s LSTM for Helpdesk

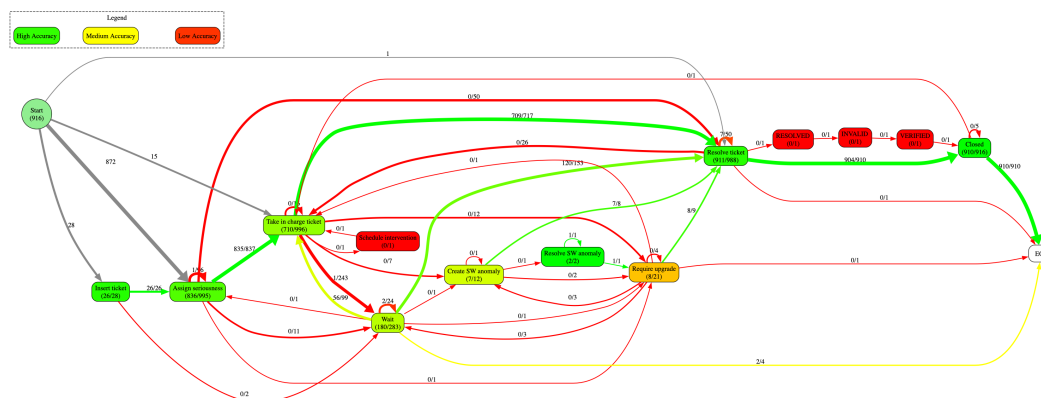


two occurrences in the test set do not allow a meaningful statement about the underlying logic. The model additionally distinguishes between multiple successors for *Wait*: it predicts the successors *Resolve ticket* and *Take in charge ticket* correctly more often than not, and thereby reaches a higher accuracy than a majority-path collapse would yield.

A further look at the *Assign seriousness* gateway shows the collapse to the majority path. The model predicts *Take in charge ticket* for nearly every prefix ending with *Assign seriousness*, which yields 835 out of 837 majority cases correct, but zero correct predictions for the rarer successors *Resolve*

Figure 23

Process Prediction Map for Camargo et al.'s LSTM on Helpdesk



ticket (0 out of 50), *Wait* (0 out of 11), and *Require upgrade* (0 out of 1). The overall accuracy of 836 out of 995 (84%) corresponds almost exactly to the majority successor path of 837 out of 995 derived in the previous section. The underlying decision logic at this gateway has therefore not been learned by this model either, but unlike the U-ED-LSTM, which fell below the majority baseline at 54%, Camargo et al.’s LSTM collapses to a more favorable default.

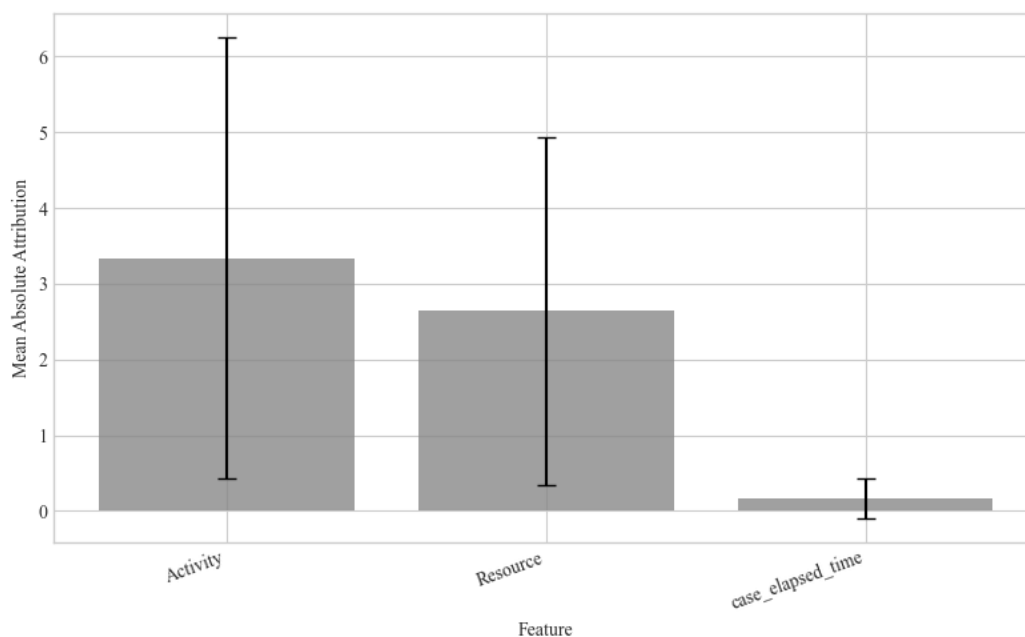
The *Wait* gateway, by contrast, is the one place where Camargo et al.’s LSTM achieves a generalization which achieves a higher accuracy as the majority collapse. Out of 283 prefixes ending in *Wait*, the two *dominant* successors are *Resolve ticket* (153 cases) and *Take in charge ticket* (99 cases), and the model predicts both correctly more often than not: 120 out of 153 (78%) for *Resolve ticket* and 56 out of 99 (57%) for *Take in charge ticket*. The resulting overall accuracy of 180 out of 283 (64%) exceeds the majority path’s accuracy of 153 out of 283 (54%), which would result from always predicting the most common successor. Coverage of the remaining branches is mixed: the self-loop is predicted correctly in only 2 out of 24 cases (8%), and the end-of-sequence prediction in 2 out of 4. The three rare successors *Require upgrade*, *Create SW anomaly*, and *Assign seriousness* each occur once and are not predicted correctly.

To analyse how the model may have arrived at its predictions, we turn to surrogate models to replicate the model’s decision using decision trees. However, the LSTM surrogate decision tree only achieves a 74% fidelity to the model, which does not suffice to make it a trustworthy source of information. Fitting a ground-truth decision tree to uncover an underlying pattern of which activity succeeds *Wait*, using only the features that the model also has access to, does not help either: it reaches only 58% accuracy.

At almost every other gateway in the Helpdesk process, the ground-truth decision tree collapses to a single node predicting the majority successor. None of the available input features (activity, resource, and elapsed case time) yields a split that significantly overachieves the majority prediction. The features available at these gateways therefore carry only limited signal for distinguishing the non-majority successors. This has direct implications for any attempt to improve the model: our analysis shows no clear path to improved performance through data oversampling or augmentation. A meaningful improvement at these gateways would require the inclusion of additional data points in the feature set.

Figure 24

Aggregated Integrated Gradients of Camargo et al.'s LSTM for BPIC-17



BPIC-17

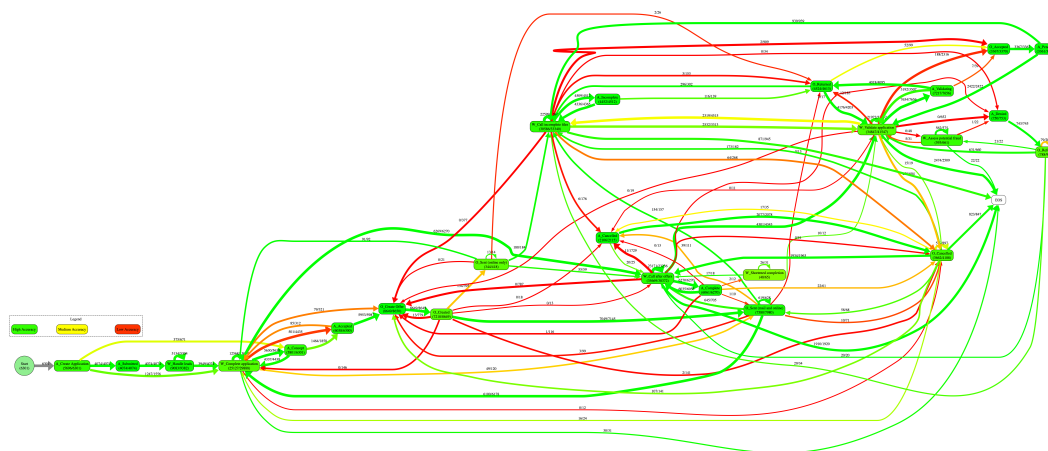
The aggregated Integrated Gradients for Camargo et al.'s LSTM on BPIC-17 (Figure 24) again attribute almost all of the signal to the activity (3.48) and the resource (2.64). The elapsed case time (0.18) is negligible. The resource carries much more weight than on Helpdesk, at roughly three quarters of the activity attribution rather than one third.

The process prediction map for Camargo et al.'s LSTM on BPIC-17 (Figure 25) shows substantially more nuanced decision behavior than on Helpdesk. At many gateways the model correctly distinguishes multiple non-trivial successors, including *A_Cancelled*, *A_Denied*, *A_Concept*, *A_Validating*, and *W_Handle leads*, which suggests that more than the majority selection has been learned on this dataset.

However, the model still fails on several paths which are the second most common at their gateway. Notable is *W_Validate application* to *A_Denied*, which occurs 652 times in the test set yet is never predicted correctly. But also at *W_Call after offers* to *A_Cancelled* (11 out of 1,729), *W_Call after offers* to *O_Create Offer* (0 out of 787), *W_Call incomplete files* to *O_Accepted* (2 out of 909), *W_Complete application* to *A_Accepted* (501 out of 4,435), and *O_Created* to *O_Create Offer* (13

Figure 25

Process Prediction Map of Camargo et al.'s LSTM on BPIC-17



out of 779). The dominant transition at each of these gateways is predicted correctly, but the model is not able to correctly predict less common paths there.

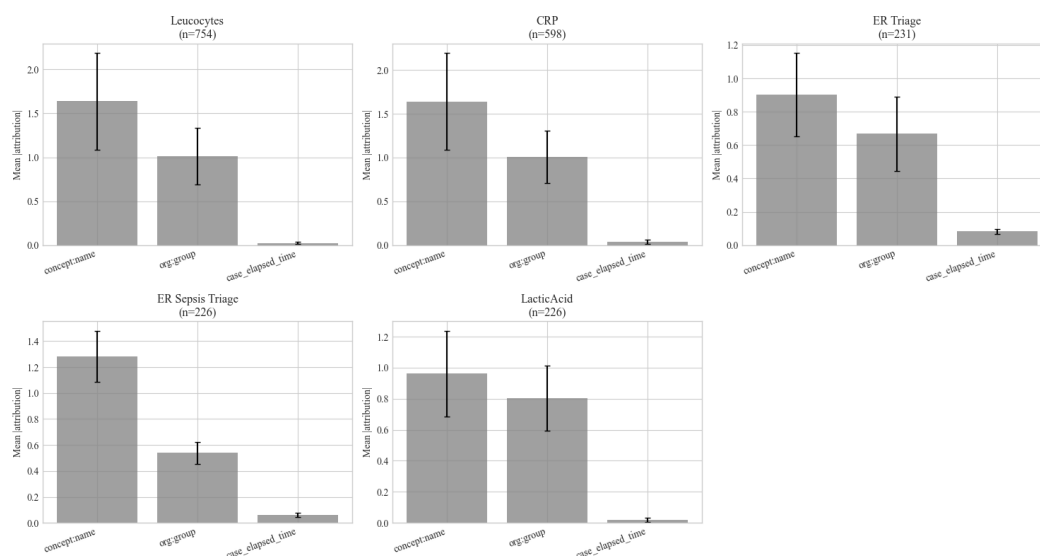
To assess whether the LSTM has missed any simple decision rules recoverable from the available input features, we fitted at each gateway ground-truth surrogate decision trees, and compare their accuracy to the LSTM's. It shows that at essentially every gateway in the process the LSTM is at least as accurate as the surrogate, often significantly more. Only in two case the surrogate is fractionally ahead, *O_Created* and *A_Complete*, differ by less than 0.5 percentage points, while the largest accuracy gains for the LSTM are at *W_Validate application* (0.84 vs 0.58) and *W_Complete application* (0.84 vs 0.47).

Two effects explain why the LSTM matches or exceeds the surrogate. First, the LSTM appears to have already learned the simple decision rules that a C4.5 tree could detect, so any signal the tree can extract from the three input features has likely also been picked up on by the LSTM. Second, the LSTM has much larger learning capabilities than the surrogate: it can represent continuous decision boundaries, feature interactions, and longer-range dependencies in the 15-event input window that a size-bounded tree can not. The large gaps at *W_Validate application* (0.84 vs 0.58) and *W_Complete application* (0.84 vs 0.47) most likely come from the latter.

This analysis does not suggest any clear improvement at these gateways through data augmentation or oversampling, since the simple decision rules recoverable from the three input features are already

Figure 26

Per Gateway Aggregated Feature Attribution Scores of Camargo et al.'s LSTM for Sepsis

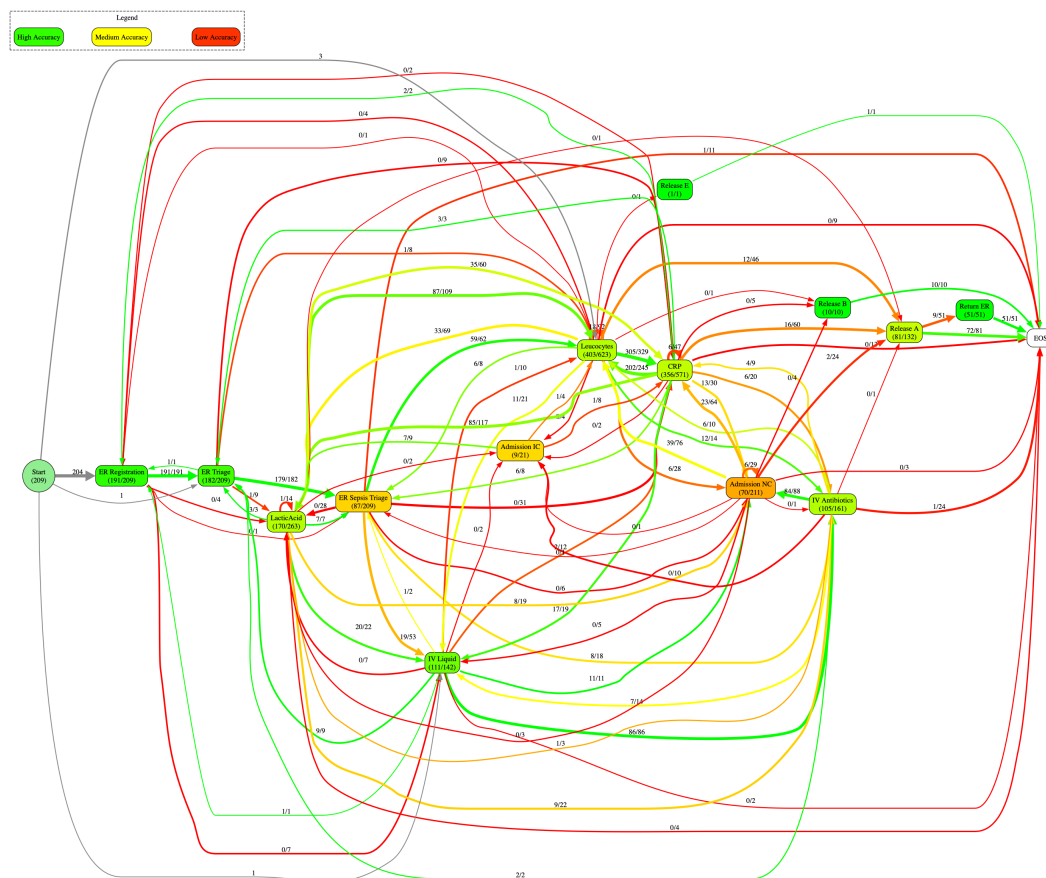


captured by the LSTM. Other improvements may still exist. A richer feature set in particular could raise the model's performance.

Sepsis

We start our analysis by looking at the feature attribution using aggregated integrated gradient scores, broken down by prediction target as shown in Figure 26. It shows that the model primarily attributes its predictions to the activity feature, and to a lesser extent to the resource feature. The case elapsed time, however, is not influencing the prediction meaningfully. The same pattern holds for *LacticAcid* predictions, where activity dominates and resource trails. For *ER Sepsis Triage* the gap is wider, with activity driving the prediction almost alone and the resource contributing little. The near-zero attribution of *case_elapsed_time* raises concerns, as it is a conceptually relevant signal that carries information on how the process is progressing. The model relies only on activity and resource, but in Sepsis the resource (*org:group*) is essentially determined by the activity. Lab tests (*Leucocytes*, *CRP*, *LacticAcid*) are always performed by group B, triage always by group C, releases always by group E, and IV interventions by group A in about 95% of cases. The two admission activities are the only exception, with *Admission NC* spread across 18+ wards and *Admission IC* across 4. For every other activity, the two features collapse into one signal and this puts into doubt whether the model has learned nuanced process behavior or has simply memorized the most common activity sequences.

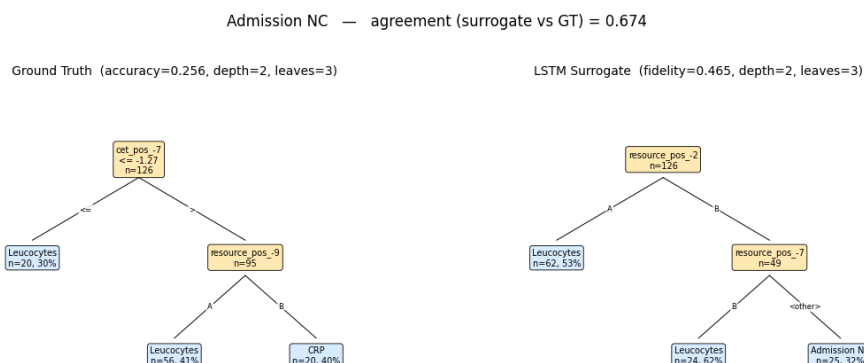
Figure 27
Process Prediction Map of Camargo et al.'s LSTM on Sepsis



The process prediction map, while relatively cluttered due to the large amount of activities, shows a set of green nodes with points where the next activity prediction is high and a few orange nodes. It's notable that the model does not show any red, starved nodes, showing the models ability to learn behavior even of rare activities. This is underlined by activities such as *Release E* which shows the model despite being a rare activity, having learned the successor correctly. Another instance which shows the models ability to learn rare variants can be seen right at the beginning. There is a very limited amount of cases, which do not begin with the activity *ER Registration* and instead begin with either another registration step like *ER Triage*, a treatment such as *IV Liquid* or a measurement taken such as *LacticAcid*. However in these cases the patient still needs to be registered which is represented in the process to be going back to the *ER Registration* activity. In 4/5 of test cases this is correctly predicted after the medical interventions were concluded. As an example this can be seen in the prediction from *ER Triage* to *ER Registration*. This is exactly the case which starts with triage instead of registration. Same for the case in which as a first measure the *IV Liquid* took place. To

Figure 28

Decision Trees for Camargo et al.'s LSTM on Sepsis at Gateway "Admission NC"



check whether the model uses the resource feature where it actually matters, we look further at the two admission gateways, which are the only places where the *Resource* carries information beyond just grouping a set of activities (In most cases, an activity is always done by the same resource). *Admission IC* occurs in only 21 test transitions, which is too rare to generalise from, so we focus on *Admission NC*. The process prediction map shows the model predicting the correct next successor in only 70 out of 211 cases (33%). The model does not collapse to the majority successor. Instead it spreads its predictions across *Leucocytes* (39/76, 51%), *CRP* (23/64, 36%), the self-loop (6/29, 21%), and a few other minor branches. The differentiation does not help, however. A baseline which always predicts *Leucocytes* would score 76/211 (36%), and the model performs below it.

To find out whether the gateway has a recoverable decision rule, we fit two decision trees on the features available to the LSTM, one as a surrogate to the model's predictions and one on the ground truth (Figure 28). The surrogate reaches a fidelity of 47%, and the ground-truth tree reaches 26% accuracy, which is also below the majority baseline. Extending this comparison across all gateways (Table 4), the LSTM matches or exceeds the tree at every gateway, with the largest gaps at *LacticAcid* (0.70 vs 0.40), *Leucocytes* (0.64 vs 0.49), and *IV Antibiotics* (0.76 vs 0.61). At both admission gateways the LSTM and tree are tied, with neither beating the majority baseline, which confirms that none of the models with these features can do better at those gateways.

We have shown earlier that the model relies on the activity and resource features, and that it fails to take advantage of the resource at the gateways where the resource actually carries information

beyond the activity. The logical conclusion is that the model bases its decisions on activity sequence patterns alone. This in itself is a valuable finding, as it exposes a clear limitation of the model.

Table 4

Per-gateway Accuracy of Camargo et al.'s LSTM, a Ground-truth Decision Tree and a Surrogate Model

Gateway	N	LSTM acc	GT tree acc	Surrogate fidelity
ER Registration	209	0.914	0.881	0.881
ER Triage	209	0.871	0.881	0.881
ER Sepsis Triage	209	0.416	0.452	0.333
IV Liquid	142	0.782	0.828	0.759
Leucocytes	623	0.647	0.640	0.488
CRP	571	0.623	0.530	0.409
LacticAcid	263	0.646	0.698	0.396
IV Antibiotics	161	0.652	0.758	0.606
Admission NC	211	0.332	0.256	0.256
Release A	132	0.614	0.741	0.630
Admission IC	21	0.429	0.400	0.400

Domestic Declarations

Camargo et al.'s LSTM receives the three features *Activity*, *Resource* and *case_elapsed_time* as input. The aggregated mean Integrated Gradients show that the model attributes its prediction primarily to the activity (0.96) and to a much lesser extend to the resource (0.45). The *case_elapsed_time* however gets hardly any attribution at 0.01.

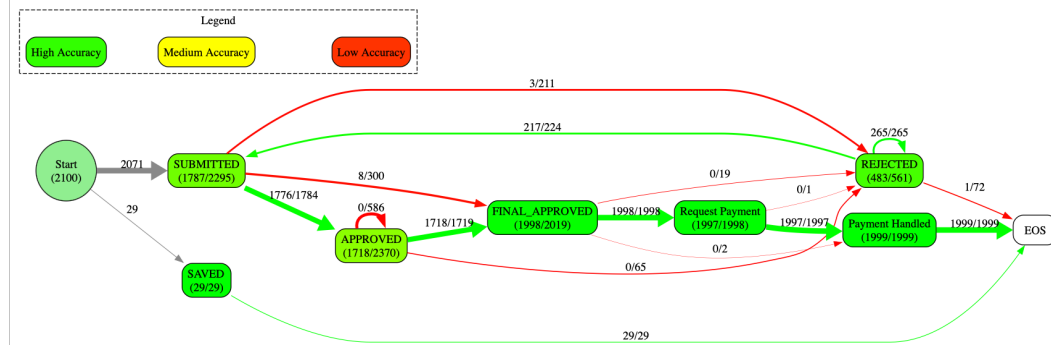
In the next step we analyse the process prediction map shown in Figure 29. This provides us with a very clear explanation of the model behavior. The model overfits to the majority successor at every gateway as can be seen by every node having one outgoing flow representing almost perfect accuracy and all the other with hardly and in many cases no correct predictions.

8 Framework

In the following we introduce a concise framework that enables developers of predictive process monitoring models to leverage explainable AI methods to *debug* their models. The framework consists of a set of Signals, which are each a triplet of an Observation, a Diagnosis, and an Action. The Observation is a pattern surfaced by one of the previously introduced explainability methods that suggests the model is performing suboptimally. The Diagnosis is the most plausible interpretation

Figure 29

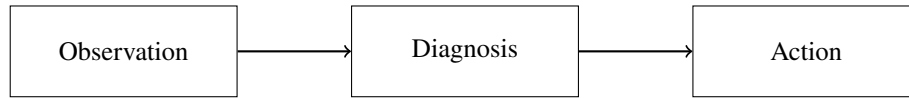
Process Prediction Map for Camargo et al.'s LSTM on Domestic Declarations



of the underlying issue that caused the observed pattern. The Action is a concrete intervention that targets the diagnosed cause and, when the diagnosis holds, makes the observed pattern disappear. Applying the Action in Chapter 8.3.2 and checking whether the Observation disappears therefore also verifies the Diagnosis. The framework's goal is to detect patterns that signal underperformance of the model and to provide Actions for closing those performance gaps and by that improving the models performance.

Figure 30

The Three Steps of the DeepBug Framework



The framework provides hypotheses, in the form of diagnoses, which are therefore the most plausible explanation, meaning they may not be the correct or only explanation. The corresponding actions therefore serve as a test. If they work at mitigating the signal, in return the diagnosis is likely to be at least in part accurate.

The framework is structured around three lenses: data set level, process prediction map, and gateway surrogates.

8.1 Data Set Level Aggregation

Signal 1.1: Low-attribution features despite semantic relevance

Observation The features with high aggregated attribution scores should usually match those that domain knowledge marks as valuable. However, some features show near-zero attribution despite being semantically relevant to the prediction task.

Diagnosis The most plausible cause is that the model has failed to use these features. Either the training data lacks sufficient variance in them, or a few dominant features have absorbed the predictive signal.

Action Augment the training data with cases in which the underrepresented features are predictive, or reduce the influence of features that currently dominate the loss, for example by masking them.

Signal 1.2: Highly-attributed feature unavailable at inference

Observation A feature receives consistently high aggregated attribution scores, yet it is either unavailable at prediction time in deployment, or it is generated by the same process tool that produced the training data and therefore correlates artificially with the prediction target. The variant index is a typical example: it is assigned by the process mining tool only after a case has been completed and is therefore unavailable while the case is still running.

Diagnosis The model has learned to rely on a feature that constitutes information leakage. The high attribution does not reflect a meaningful predictive pattern, but a shortcut through a feature that encodes the answer either directly or through correlation with a downstream artefact. Aggregated test-set attribution is misleading in this case, since the leakage feature looks indistinguishable from a legitimately important one until its provenance is checked.

Action Mask the feature during training, or remove it from the input set entirely, and retrain the model. If the feature carries genuinely useful signal in some deployment scenarios but not in others, dropout-style masking during training encourages the model to generalize beyond it while keeping its availability where applicable.

8.2 Process Prediction Map

Process mining offers a range of algorithms for deriving process representations, such as Petri nets or directly-follows graphs. Overlaying these representations with the predictions of PPM models, and coloring them by prediction quality, makes patterns of lacking and strong performance directly visible. Three signals operate at this level, each tied to a distinct visual pattern in the prediction map.

Signal 2.1: Gateway majority collapse

Observation A node has one green outgoing flow while all other outgoing flows are red. The model therefore always predicts the same successor, regardless of the prefix that ends at this node.

Diagnosis The model has collapsed a multi-branch decision point into a majority-class default and predicts the most frequent successor irrespective of contextual features. This points to either class imbalance at that gateway (one successor dominates the training distribution) or insufficient feature signal for the model to distinguish the minority branches.

Action We begin with a second diagnostic check: fit a surrogate on the same gateway. If the surrogate also collapses to the majority, a lack of information in the features cannot be ruled out as the cause, and Signal 3.2 applies instead. If it achieves higher accuracy with simpler decision logic, the model has learned a suboptimal pattern, and the surrogate's rules supply an inductive bias for retraining (see also Signal 3.1). To shift the model towards the more favorable pattern, rebalance the model's exposure to the minority branches, for example by oversampling or augmenting prefixes that lead to those successors.

Signal 2.2: Below-majority misinterpretation at a gateway

Observation At a gateway, the model performs worse than a trivial majority-class baseline would, even though the activity at the gateway is not rare in training. The model does not collapse to the majority successor, but actively predicts a wrong successor that does not correspond to the most common path either.

Diagnosis The model has likely learned an association at this gateway, just the wrong one. A common case in process data is that it treats an inserted activity as a replacement for a neighboring activity and predicts the neighbor's successor instead of the neighbor itself. The same failure can also surface as the model falling back on the global activity distribution at the gateway, predicting an activity that is frequent elsewhere in the event log but rarely correct at this specific decision point. Either way, the model substitutes the gateway's local successor distribution for a different one.

Action For the global-distribution case, raise the weight of the gateway’s local pattern in the training signal until it outweighs the dataset-wide pattern the model is currently using. Available techniques include oversampling the not-learned cases, reweighting the loss at specific gateways, or reweighting classes on the head. For the neighbor-replacement case, broaden the contexts in which the misinterpreted activity appears in training so its role becomes unambiguous from the surrounding control flow. A useful method for determining a threshold of the rebalancing is the process prediction map itself. By looking at the nodes, where the model has learned a successor correctly and the ones, where all the successors are predicted incorrectly one can infer a threshold at which outlier events become common enough to be picked up by the model.

Signal 2.3: Invisible activities from class starvation

Observation A node has only red incoming flows. The model never predicts this activity when it is the true successor, and likely never predicts it at all.

Diagnosis The activity is too rare in the training data for the model to have learned when to predict it. The model defaults to more frequent alternatives at every decision point where this activity could appear.

Action Oversample prefixes that lead to the starved activity, or generate synthetic variants through suffix-preserving augmentation. Often, however this symptom is not worth addressing, as they refer to outlier cases.

8.3 Gateway Surrogates

To explain the model’s decision behavior at gateways, intrinsically interpretable surrogate models (e.g. decision trees) are trained both to mimic the model’s predictions and to recover the ground-truth decision logic from the available features. Comparing the model, the model-mimicking surrogate, and the ground-truth surrogate reveals where the model’s learned logic diverges from the actual process, and whether the available features even carry enough signal to support a better prediction. Two signals operate at this level.

Signal 3.1: Surrogate outperforms the deep learning model

Observation A shallow decision tree trained on the ground truth labels at a gateway achieves higher accuracy than the deep model, despite having far fewer parameters and no access to sequential context.

Diagnosis The gateway’s decision logic is simple enough for a tree to capture, yet the model has learned a suboptimal or incorrect mapping. The cause is typically one of three: class imbalance suppressing the minority branches, reliance on spurious features that do not generalise, or insufficient training exposure to the feature combinations that govern this gateway.

Action Extract the tree’s decision rules to identify which features and thresholds separate the successors correctly. Compare these to the model’s IG attributions at the same gateway: if the model overweights features that are absent from the tree, this suggests false reliance. Concrete remedies include augmenting the underrepresented branches, or a hybrid setup in which the surrogate handles this specific gateway while the deep model handles the more complex parts of the process.

Signal 3.2: Ground-truth surrogate also collapses to majority

Observation At a gateway where the deep model collapses to the majority successor, a ground-truth decision tree fitted on the same input features as the model also collapses to the majority. Neither the model nor the surrogate beats a trivial baseline.

Diagnosis The discriminative signal needed to choose between the gateway’s successors is not contained in the available input features. This is a feature-set problem, not a model problem, and no amount of data augmentation, loss reweighting, or capacity tuning will resolve it as long as the feature set stays the same.

Action Enrich the feature set with attributes that are likely to carry decision-relevant signal at the gateway. Candidates include longer prefix windows, contextual attributes from the case (such as resource workload or case-level metadata), or domain features known to influence the relevant

business decision. Before retraining the model, the surrogate should be re-evaluated on the enriched feature set, since this confirms that the new features actually carry the missing signal.

Table 5

Framework Signals and Model in which they Appear

Signal	Appears in
1.1 Low attribution, semantic relevance	U-ED-LSTM > Sepsis
1.2 Inference-time leakage	U-ED-LSTM > Helpdesk
2.1 Gateway majority collapse	U-ED-LSTM > Sepsis; U-ED-LSTM > DD; Camargo > Helpdesk
2.2 Below-majority misinterpretation	U-ED-LSTM > Helpdesk; U-ED-LSTM > Sepsis; Camargo > Sepsis
2.3 Invisible activity (starvation)	U-ED-LSTM > Helpdesk
3.1 Surrogate outperforms model	U-ED-LSTM BPIC-17
3.2 Ground-truth surrogate also collapses	U-ED-LSTM > Helpdesk; U-ED-LSTM > Sepsis; Camargo > Helpdesk

Evaluation

In this Chapter we assess the framework introduced earlier (Chapter 8) by applying appropriate actions to signals that have been identified in the analysis Chapter and analyze if this remedied the identified shortcomings of the models. We evaluate this on multiple levels. It starts at the per-gateway decision, if the model's performance has changed or improved at particular points in the process. It then checks whether this has translated into better bottom-line performance for next-event prediction on the test set. And lastly, since the U-ED-LSTM in particular is designed for suffix prediction, we check whether the improvement trickled down to suffix-level metrics.

The evaluation is applied to the datasets: Helpdesk, BPIC-17 and Sepsis, while Domestic Declarations is out of scope for this part of the thesis.

8.4 Uncertainty-aware Encoder-Decoder LSTM (U-ED-LSTM)

Helpdesk

Signal 1.2: Variant Index as Information Leakage. The data-set level lens (Signal 1.2) flagged the feature "Variant index" as a possible case of information leakage. It receives the second-highest aggregated attribution score among all input features, yet from domain understanding it is unclear whether the feature would be available in a production environment. What the framework contributes here is the detection itself and the ability for the model provider to double-check the

input feature set. As removing the feature would fundamentally shift the model's behavior and limit comparability with the original model, we abstained from doing so at this point.

Signal 2.2: Oversampling Experiments at "Insert ticket" and "Assign seriousness".

We ran two oversampling experiments to address the Signal 2.2 instances from the analysis: the "Insert ticket" misinterpretation, where the model predicts "Take in charge ticket" as if "Insert ticket" replaced "Assign seriousness," and the "Assign seriousness" gateway, where the model fell below the majority baseline.

Signal 2.2 recommends augmenting the training data so that the misinterpreted activity appears in different contexts, giving the model more data from which to learn the underlying control flow. We could not apply this directly on Helpdesk because of the "Variant index" feature, which records the case-level process variant. Inserting *Insert ticket* into cases that originally start with *Assign seriousness* would produce activity sequences whose "Variant index" no longer matches any real combination in the log, which would mean training the model on impossible variant-process combinations. The first experiment is therefore a simpler form of Signal 2.2's recommendation, run to test whether raising frequency alone might resolve the issue. Cases starting with *Insert ticket* were oversampled by a factor of 40, raising their frequency without changing the contexts in which *Insert ticket* appears. The factor takes the *Insert ticket*-start path roughly to the frequency level of the dominant "Assign seriousness"-start path, which is about 37 times more common in the log (4384 vs 118 cases). The intervention did not improve the predictions at the "Insert ticket" prefix: the model continues to incorrectly predict "Take in charge ticket" instead of "Assign seriousness" in 100% of the cases. Oversampling only repeats prefix-target combinations that the model has already learned, including the incorrectly predicted "Insert ticket" to "Take in charge ticket" flow, and never shows the model an "Insert ticket" prefix whose successor differs from "Take in charge ticket." The negative result therefore points to augmentation and perturbations as the part of Signal 2.2's recommendation that addresses this misinterpretation, which can only be applied properly once the "Variant index" feature is removed, or at least masked partially.

The second experiment applied an oversampling factor of 2 to all cases with prefix length one, giving the model more exposure to early-process predictions across all starting activities. At "Assign seriousness," accuracy rose from 54% to 79%, closing most of the gap to the 84% majority baseline. Signal 2.2's recommended action also increases the minority-branch accuracy, though

only marginally, so the improvement is more than a collapse to the majority. Before the intervention, the model's most frequent prediction at "Assign seriousness" was "Resolve ticket," a rare minority successor at this gateway rather than the dominant one. After the intervention, that collapse to a minority successor is gone. The model now predicts the actual majority branch "Take in charge ticket" on most cases, and correctly predicts a small share of the genuine "Resolve ticket" minority cases. The other rare branches at the same gateway, "Wait" and "Require upgrade," remain unpredicted.

Beyond these two experiments, there is more information to take from the explainability analysis, such as which predictions the model gets right and at which gateways, learning where the model can be trusted and where not. Extracting direct actions for improvement from those further observations may be more difficult or borderline impossible given the feature set at hand, the model architecture, the training sample size, and so on. We have, however, shown that it is possible to get measurable increases in accuracy clearly traced back from the XAI analysis via the framework.

Results. Beyond the gateway-level effects, the prefix-length-1 oversampling produced clear gains on the full test-set metrics for the U-ED-LSTM on Helpdesk (Table 7). Next-event accuracy rose from 67.74% to 74.93%, the Damerau-Levenshtein Similarity (DLS) from the most-likely suffix prediction 0.8315 to 0.8596, and for the probabilistic sampling using 100 samples, the DLS increased from 0.6794 to 0.7377. Thus all three metrics moved in the expected direction.

BPIC-17

The analysis Chapter described three issues for the U-ED-LSTM on BPIC-17, which we here map to framework signals. At *A_Denied* the model predicts globally popular activities *W_Validate application* and *W_Call incomplete files* instead of the local, gateway-specific successor, which in this instance is *O_Refused* in 99% of cases. This corresponds to Signal 2.2 (below-majority misinterpretation at a gateway). At *A_Create Application*, the model fails to predict the rare *Limit raise* to *A_Concept* branch despite the gateway's logic being learnable from the inherently interpretable decision tree. This matches Signal 3.1 (surrogate outperforms the deep learning model).

Signal 2.2: A1 case oversampling. Signal 2.2's action is to rebalance training so the gateway-dependent distribution outweighs the dataset-wide frequencies. We oversample every training case in which *A_Denied* occurs, choosing the factor so the augmented frequency of *A_-*

Denied matches that of activities whose successors the process prediction map shows as learned correctly. That frequency is the threshold above which the model picks up a gateway's local pattern on this dataset and model. For *A_Denied*, this is a factor of seven.

Signal 3.1: A2 case oversampling. For the Limit Raise branch, Signal 3.1 points to upsampling the underrepresented branch. We duplicate the cases that take the Limit Raise branch by a factor of four, which lifts Limit Raise to a frequency comparable to each of the two New credit branches at this gateway. Since the model already predicts the New credit branches correctly, their frequency serves as an indication for the level at which the gateway's local pattern enters the model's training signal.

Results. The results of the measures were mixed. The *A_Denied* successor prediction has improved. There we moved from a correct successor prediction accuracy of less than 1% to 94% (713 of 755). On the other hand, the oversampling of the Limit Raise branch has been counterproductive. The accuracy at the *A_Create Application* gateway has decreased from 87% to 12% instead of improving. Overall this led to a decrease in next event accuracy of 14pp and reduction of the DLS metrics (for probabilistic sampling and the most-likely suffix) of 0.04. This suggests that the augmentation resulted in a shift in the training data that led the training procedure to become unsuitable. This shows a weakness of the framework, which addresses issues one at a time without ensuring the holistic soundness of the model and training procedure.

Sepsis

Ahead of applying the measures for the framework signals, the process prediction map (Figure 19) suggested a general issue with the training procedure, showing starved activities across the whole process. Instead of addressing each of them individually, this prompted us to do a hyperparameter retune before we start the evaluation of the framework driven actions. The retune is not part of the frameworks actions. We changed the hidden size from 128 to 64, the number of LSTM layers from 4 to 2, the learning rate from 1e-5 to 1e-4, the batch size from 128 to 32, and the ReduceLROnPlateau scheduler to a gentler factor 0.5 with patience 5 (instead of factor 0.01 with patience 1). The tuned configuration drove next-event prediction accuracy from 24.5% in the initial state to roughly 41.0%, an about 16 percentage-point improvement.

Signal 1.1: Masking the dominant categorical features. In the analysis Chapter we identified an underusage of relevant features, which corresponds to Signal 1.1. The medical attributes received near-zero attribution while the three process attributes (*concept:name*, *lifecycle:transition*, *org:group*) carried almost all the predictive power. Signal 1.1’s action is to mask the dominant features so the model is forced to attend to the underused ones.

We applied this using feature dropout that randomly zeros *lifecycle:transition* and *org:group* with a probability of 0.2. The activity feature *concept:name* was kept unmasked on purpose, since it is the model’s most used input and masking it would likely prevent the model from learning the core process structure. An initial attempt at a probability of 0.5 collapsed the model entirely (test accuracy of 0.018). However, reducing it to 0.2 produced a stable training run.

The overall test-set accuracy improved from 41.0% (the hyperparameter-tuned baseline) to 43.5% (+2.5 percentage points), so the masking action does help in aggregate. However, at the admission gateways where the medical features should provide the most discriminative signal, accuracy decreased as opposed to the anticipated increase. Admission NC went from 33.2% (70/211) to 29.0% (62/211), and Admission IC from 42.9% (9/21) to 31.6% (6/19).

The aggregated attribution after masking shows that the model’s aggregated integrated gradients have not changed either. The three categorical process attributes remain as the clear top (*lifecycle:transition* at 0.247, *org:group* at 0.220, *concept:name* at 0.144), and the medical features remain near zero with the highest at 0.024 (*DiagnosticSputum*). The masking action therefore improved the model’s next-event prediction accuracy but did not follow the causal connection described in Signal 1.1 of the framework.

Signal 2.2’s action is to rebalance training so that gateway-conditional distributions carry more weight than dataset-wide frequencies. Rather than oversampling at the prefix level, we applied this at the loss level. The two are equivalent in expectation, but loss-level weighting is a one-line change to the cross-entropy rather than a restructuring of the data pipeline. The cross-entropy on the *concept:name* head was reweighted so that rare activities trigger larger gradients. Each class c received a weight $w_c \propto (1/n_c)^{0.5}$, where n_c is the number of times class c appears in the suffix positions of the training set. Frequent classes get smaller weights, rare ones get larger weights. Plain

inverse-frequency ($T = 1.0$) would have overweighted the rarest classes too much. Using $T = 0.5$ produced a more subtle balance.

Adding the weighted CE on top of the tuned training procedure produced no measurable improvement over the tuned baseline (41.03% vs 40.96%). This is a null result for Signal 2.2's action on this model and dataset.

Signal 3.2: Limited signal in the event log. Signal 3.2's action is to extend the input when the available features lack discriminative signal through the inclusion of more features. The U-ED-LSTM on Sepsis already receives the full set of attributes that the event log provides, unlike for Camargo. This means it is not possible here. However, the Signal 1.1 finding, i.e. near-zero attribution on the medical attributes, and the masking test together suggest that the existing features genuinely do not carry enough discriminative signal at these gateways. Signal 3.2's action of additional feature enrichment needs to come from beyond the provided event log. We defer this to future work.

Results. On the full test set the retrained model improves on every metric (Table 7). Next-event accuracy rises from 24.49% to 43.49%. The improvement carries over to the suffix-prediction metrics: probabilistic suffix-sampling DLS, averaged over 100 sampled suffixes per prefix, rises from 0.17 to 0.23, and the most-likely suffix similarity rises more strongly, from 0.12 to 0.30.

8.5 Camargo et al.'s LSTM

Helpdesk

Signals 2.1 and 3.2: Performance Ceiling at Most Gateways. The analysis identified Signal 2.1 (gateway majority collapse) at almost every multi-branch gateway in the Helpdesk process for Camargo et al.'s LSTM. The *Wait* gateway is the one exception, where the model performs above the majority baseline and correctly distinguishes between the gateway's successors. Signal 2.1's action starts with a diagnostic check, fitting a surrogate on the gateway and seeing whether it collapses too. It does at every collapsed gateway. A ground-truth surrogate trained on the same three input features (activity, role, elapsed case time) also defaults to the majority successor, so the diagnosis moves to Signal 3.2, which reads this as the available features missing the discriminative signal needed to choose between the gateway's successors. As Signal 3.2 itself notes, no training-side intervention make up for that information as long as the feature set stays the same.

This puts Camargo et al.’s LSTM in a different position from the U-ED-LSTM on Helpdesk. The U-ED-LSTM had a below-baseline gap at *Assign seriousness* that prefix-length-1 oversampling could close with more training signal at length-1 prefixes. Camargo et al.’s LSTM has no such gap. It sits at or near the majority baseline at every collapsed gateway and above it at *Wait*. On the same dataset, the framework’s Signal 2.1 / 3.2 routing therefore identifies where headroom exists in the current feature set, here U-ED-LSTM, and where it does not, here Camargo et al.’s LSTM.

We did not apply Signal 3.2’s recommended action in this work, since the model’s feature selection was a deliberate choice by its original authors. The action would be to increase the feature set with the other event log attributes. It is nevertheless the natural next step for improving performance, as no other clear area for improvement could be identified through our analysis.

BPIC-17

Signal 3.2: Feature-Set Ceiling Above the Surrogate. Signal 3.2 in the framework strictly describes a gateway at which both the model and the surrogate collapse to the majority. The surface observation for Camargo et al.’s LSTM on BPIC-17 differs from this, since the model is not collapsing at all. It distinguishes multiple non-trivial successors at *A_Cancelled*, *A_Denied*, *A_Concept*, *A_Validating*, and *W_Handle leads*, and at most gateways it beats a ground-truth decision tree fitted on the same input features. The largest gaps between the LSTMs and surrogates accuracy reach 0.84 vs 0.58 at *W_Validate application* and 0.84 vs 0.47 at *W_Complete application*. At two gateways (*O_Created* and *A_Complete*) the surrogate is fractionally ahead, but by less than 0.5 percentage points each, which is not enough to count as a Signal 3.1 (Surrogate outperforming deep model) case.

The remaining errors come from non-majority branches of a set of gateways, like *W_Validate application* -> *A_Denied* (the second-most-common branch, 0 out of 652 correct) and *W_Complete application* -> *A_Accepted* (the third-most-common branch, 501 out of 4435). The surrogate cannot recover a logic at those branches either, which suggests that either the LSTM has already exhausted what the three input features (activity, resource, case elapsed time) can encode at these gateways or that the model’s limitations are reached.

To test however, if the model can capture more accurate decision logic, when receiving an extended set of input features would be the logical next step. However, since the feature selection was a deliberate decision by the authors, this will be noted as possible future work.

Sepsis

Signal 1.1: Underused `case_elapsed_time` and resource redundancy. The data-set-level lens flagged that `case_elapsed_time` shows a Signal 1.1 case, which describes that an input feature has lower attribution scores than would be expected per domain understanding. In this case, `case_elapsed_time` has a near-zero attribution. Further analysis also showed that the resource attribute is essentially determined by the activity for every activity outside the two admission events, so the model effectively trains only on the activity sequence.

Signal 1.1 suggests masking the dominant feature so the model has to use the underused one, or augmenting the data so it appears more during training. Neither really fits here. The input set is small to begin with: activity carries most of the predictive signal, resource is mostly redundant with activity, and time is just a single scalar. Masking activity would leave the model with the redundant resource and the time, which is not enough to learn useful predictions. Augmentation does not fit either, since time is present in every training case and there is no specific group of cases where it is more predictive than others. The Signal 3.2 action below, which suggests using an increased input set with new features, addresses the main issue.

Signals 2.2 and 3.2: Feature-set ceiling at the admission gateways. The analysis Chapter identified *Admission NC* as a gateway where the model predicts worse than the collapse to the majority baseline would (Signal 2.2), and showed that a ground-truth decision tree on the same input features also fails to beat the majority baseline there. Signal 3.2 strictly applies when the deep model itself collapses to the majority, which is not what happens here, but the underlying problem is the one Signal 3.2 was written for: the discriminative signal needed at the admission gateways is not contained in the available features. We therefore treat this as a special Signal 3.2 case, reached from a Signal 2.2 surface observation rather than from a Signal 2.1 collapse.

Signal 3.2's recommended action is to enrich the feature set with attributes likely to carry decision-relevant signal at the gateway. The Sepsis log provides a large set clinical data that the original input feature set does not include, and which plausibly drives clinical decisions and process flows. For

Table 6
Prediction Accuracy per Gateway Before and After Adjustments

Gateway	Avg. pos	Before	After	Δ %
ER Registration	1.03	91.4%	91.4%	0.0
ER Triage	2.22	87.1%	85.2%	-1.9
ER Sepsis Triage	3.55	41.6%	36.4%	-5.3
IV Liquid	6.13	78.2%	75.4%	-2.8
IV Antibiotics	7.39	65.2%	57.8%	-7.5
Admission IC	10.48	42.9%	33.3%	-9.5
LacticAcid	10.70	64.6%	62.7%	-1.9
Admission NC	11.43	33.2%	32.7%	-0.5
CRP	13.20	62.3%	63.7%	+1.4
Leucocytes	13.98	64.7%	65.5%	+0.8
Release A	16.27	61.4%	64.4%	+3.0
Release B	17.40	100%	100%	0.0
Return ER	17.56	100%	100%	0.0
Release E	23.00	100%	100%	0.0
Overall		64.9%	64.2%	-0.7

example the ward a patient is sent to is determined by which body parts are to be examined. Thus we retrained Camargo et al.'s LSTM with these attributes added to the input, while keeping the old training procedure unchanged.

Signal 2.2's other action, designing an augmentation that forces the correct interpretation, is unlikely to improve performance on its own here. The surrogate has shown that no model with the existing features can beat majority at this gateway, so augmentation alone would just generate more cases the features cannot drive better decisions.

Results. Overall test-set accuracy moved from 64.9% before the change to 64.2% after, a slight decrease. The per-gateway-activity breakdown in Table 6 shows a more structured pattern than the aggregate suggests. When the activities are ordered by their average position in the process, every activity in the first half either loses accuracy or stays flat, and every activity in the second half either gains accuracy or stays at the 100% ceiling. The split falls around the admission gateways, which sit at average positions 10.5 and 11.4. The largest losses are at *IV Antibiotics* (-7.5 pp) and *Admission IC* (-9.5 pp). The largest gains are at *Release A* (+3.0 pp) and *CRP* (+1.4 pp).

Signal 3.2's action was aiming to give the model relevant information at the admission gateways specifically, but neither *Admission NC* (-0.5 pp) nor *Admission IC* (-9.5 pp) improved. The pre-

dictions which have become more accurate are those later in the process. A plausible reasoning is that the information is only just collected during the first steps for the process and thus can only be leveraged by the predictions later in the process.

Table 7

Accuracy across U-ED-LSTM and Camargo et al.'s LSTM

Model	Dataset	Version	Acc. (%)	DL _{prob} ¹	DL _{argmax}
Camargo et al.'s LSTM	BPIC-17	baseline	89.92	0.3314	0.1476
Camargo et al.'s LSTM	Helpdesk	baseline	80.44	0.7275	0.8597
Camargo et al.'s LSTM	Sepsis	baseline	64.94	0.2061	0.1722
Camargo et al.'s LSTM	Sepsis	retrained	63.21	0.2062	0.1836
U-ED-LSTM	BPIC-17	baseline	81.92	0.2911	0.3043
U-ED-LSTM	BPIC-17	retrained	68.22	0.2518	0.2619
U-ED-LSTM	Helpdesk	baseline	67.74	0.6794	0.8315
U-ED-LSTM	Helpdesk	retrained	74.93	0.7377	0.8596
U-ED-LSTM	Sepsis	baseline	24.49	0.1734	0.1189
U-ED-LSTM	Sepsis	retrained ²	43.49	0.2294	0.2979

9 Discussion

9.1 Findings

During the application of the framework we ran the signal-to-action pipeline end-to-end on six of the seven signals. The seventh, Signal 1.2, remained a purely informational finding. Across these six applications we achieved performance improvements on two cases, Signal 2.2 applied to the U-ED-LSTM on Helpdesk and Signal 1.1 applied to the U-ED-LSTM on Sepsis, demonstrating the framework's ability to identify areas for improvement and direct concrete corrective action. On the other cases, the framework could not produce a model with improved accuracy. On a well-tuned model, the framework does not provide directly actionable improvements within the existing model and training set, and remains descriptive in pointing out shortcomings. Achieving further performance gains in that case requires data from beyond the event log or a fundamentally different model architecture, neither of which the framework prescribes.

A separate finding concerns the process prediction map as a diagnostic tool in its own right, beyond its role within specific signals. Aggregate metrics such as next-event accuracy compress behaviour

¹Probabilistic suffix sampling uses 100 samples per prefix throughout, except for the U-ED-LSTM on BPIC-17 where we used a sample size of 25 due to resource constraints.

²Includes non-framework improvement

into a single number and can hide serious failure modes. For instance, a model that collapses to the majority successor at every gateway, or that learns one path well and ignores the rest, can still post a respectable accuracy. The process prediction map makes both visible. The per-gateway colour coding shows which gateways the model handles and which it does not, and overlaying the predictions on the DFG turns wrong predictions into visibly wrong edges. Further, this more nuanced view of the process helps identify hard limitations of the model, and possibly of the dataset itself, in terms of which decisions can plausibly be made from the data at hand. The same view also supports hyperparameter tuning alongside classical loss metrics, as demonstrated by the Sepsis retune (24.49% to 41.0%) that began from observing widespread starvation in the map.

9.2 Limitations

While the framework proved effective in several of our experiments, the evaluation also surfaced a few limitations worth noting.

The framework is designed as a set of atomic Signal-Action pairs. This structure allows a model provider to diagnose and address one Signal at a time, without requiring a global view of every other Signal or intervention. The trade-off is that the framework does not model what happens when multiple Actions interact in the same training run. A future version could extend the framework with a step that verifies compatibility between applied Actions or adds a holistic view to oversee the interlinking effects.

The second limitation concerns Signal 1.1's underlying assumption, that the model provider can correctly identify which features should be semantically relevant for the prediction task. On Sepsis we treated the medical features as the natural source of decision-relevant signal at the admission gateways, both because the dataset is organised around them and because intuition suggests they should matter. Signal 1.1's prescribed Action of masking the dominant features was therefore meant to direct the model towards them. The masking improved overall accuracy but not substantially, and the attribution distribution did not shift towards the medical input features. On Camargo Sepsis we added the medical features directly, and this did not improve accuracy at the targeted gateways either. We pick up this question in the Future Work section.

The third point is about coverage. The framework can naturally only be fully applied on models that exhibit the Signals it describes. While this was the case for six of the seven Signals across our

experiments, a stronger evaluation would apply each Signal across multiple models to confirm that the prescribed Actions generalise. And further, the Actions the framework provides must be feasible on the model and the dataset at hand. For example, if the framework suggests that underperformance is due to a lack of discriminative signal in the input features, the Action of adding features can only be carried out if additional informative features are available.

9.3 Future Work

The first item follows directly from the limitation discussed above. Signal 3.2's recommended action is to extend the input with attributes that carry additional discriminative signal. We tested this on Camargo Sepsis by adding the available medical attributes, but the change did not improve accuracy at the admission gateways. The U-ED-LSTM already uses all features in its original configuration, and even there the model's predictions barely attribute to the medical ones. The medical data points in the Sepsis event log therefore do not appear to be relied on by either model. Whether this is because the data points genuinely carry little decision-relevant signal, or because both architectures struggle to extract and combine the medical facts during process prediction, is an important question for unlocking further accuracy gains. The same question applies not just to Sepsis but to all four datasets we evaluated.

As part of the framework we provided a set of explainability methods specifically targeted for Predictive Process Monitoring, such as the Process Prediction Map. In this thesis, however, we only evaluated them in the context of model debugging and improvement, and that within a fairly rigid signal-to-action structure. Further work could explore how these methods can be helpful in other contexts. This could include a less structured debugging and model-improvement setting, covering tasks such as architecture selection and hyperparameter tuning, which we addressed only to a limited extent in this thesis. Beyond debugging, the methods could also be evaluated in terms of how they help the end user build trust in the predictions and the underlying AI system itself.

And lastly, further work can be done on the framework itself, for example by applying it to more datasets and models, expanding the signal set, and adding more concrete corrective Actions to the existing signals.

10 Conclusion

This thesis investigated how explainable AI methods can be applied to predictive process monitoring, with the goal of supporting model developers in diagnosing and improving the quality of their models. We did this by combining classical explainability techniques with new ones developed specifically for PPM models. The contribution of this work falls into four areas. We first conducted a structured review of post-hoc explainability methods in PPM, investigating the field’s current state and identifying where prior work had focused. Based on that review, we provided a set of explanation methods for PPM models that combines classical XAI techniques with new methods developed specifically for the process-monitoring setting. Integrated Gradients and SHAP were applied for feature attributions of the models. On top of these, we developed two PPM-specific methods. The Process Prediction Map which leverages process mining artifacts and techniques to build a visual tool for inspecting the model’s behavior in a way that is natural for process domain experts. The per-gateway surrogate decision trees which combine decision mining and inherently interpretable surrogate models to compare a model’s learned decision logic against ground-truth decision rules. To evaluate these methods, we developed a framework that tied actionable tasks to patterns surfaced by the explainability methods to achieve model improvements. We can therefore answer the research questions as follows: RQ 1 is answered by the literature review. RQ 2 is answered by the development and application of the three explanation methods. RQ 3 is answered by the framework and its evaluation, showing that explainability-based signals can lead to measurable performance improvements in two of the seven Signal applications, and to a deeper understanding of the model’s behavior in the others. The thesis’s contribution lies in the way it makes the model’s behavior visible, including aspects that are often overlooked during model development when focusing on aggregated accuracy scores.

Bibliography

- [1] W. Van Der Aalst, A. Adriansyah, A. K. A. De Medeiros, *et al.*, “Process mining manifesto,” in *Business Process Management Workshops*, vol. 99, Series Title: Lecture Notes in Business Information Processing, Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 169–194, ISBN: 978-3-642-28107-5 978-3-642-28108-2. DOI: 10.1007/978-3-642-28108-2_19. [Online]. Available: https://link.springer.com/10.1007/978-3-642-28108-2_19 (visited on 10/26/2025).
- [2] A. Erasmus, T. D. P. Brunet, and E. Fisher, “What is interpretability?” *Philosophy & Technology*, vol. 34, no. 4, pp. 833–862, Dec. 2021, ISSN: 2210-5433, 2210-5441. DOI: 10.1007/s13347-020-00435-2. [Online]. Available: <https://link.springer.com/10.1007/s13347-020-00435-2> (visited on 09/22/2025).
- [3] C. Rudin, “Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead,” *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, May 13, 2019, ISSN: 2522-5839. DOI: 10.1038/s42256-019-0048-x. [Online]. Available: <https://www.nature.com/articles/s42256-019-0048-x> (visited on 09/30/2025).
- [4] ISO/IEC, *ISO-IEC information technology — artificial intelligence — objectives and approaches for explainability and interpretability of machine learning (ML) models and artificial intelligence (AI) systems*, 2025.
- [5] F. M. Maggi, C. D. Francescomarino, M. Dumas, and C. Ghidini, “Predictive monitoring of business processes,” in vol. 8484, 2014, pp. 457–472. DOI: 10.1007/978-3-319-07881-6_31. arXiv: 1312.4874[cs]. [Online]. Available: <http://arxiv.org/abs/1312.4874> (visited on 10/03/2025).
- [6] B. Wuyts, S. Vanden Broucke, and J. De Weerd, “SuTraN: An encoder-decoder transformer for full-context-aware suffix prediction of business processes,” in *2024 6th International Conference on Process Mining (ICPM)*, Kgs. Lyngby, Denmark: IEEE, Oct. 14, 2024, pp. 17–24, ISBN: 979-8-3503-6503-0. DOI: 10.1109/ICPM63005.2024.10680671. [Online]. Available: <https://ieeexplore.ieee.org/document/10680671/> (visited on 10/27/2025).

- [7] E. Rama-Maneiro, J. C. Vidal, M. Lama, and P. Monteagudo-Lago, “Exploiting recurrent graph neural networks for suffix prediction in predictive monitoring,” *Computing*, vol. 106, no. 9, pp. 3085–3111, Sep. 2024, ISSN: 0010-485X, 1436-5057. doi: 10.1007/s00607-024-01315-9. [Online]. Available: <https://link.springer.com/10.1007/s00607-024-01315-9> (visited on 10/27/2025).
- [8] E. Rama-Maneiro, F. Patrizi, J. Vidal, and M. Lama, “Towards learning the optimal sampling strategy for suffix prediction in predictive monitoring,” in *Advanced Information Systems Engineering*, G. Guizzardi, F. Santoro, H. Mouratidis, and P. Soffer, Eds., vol. 14663, Series Title: Lecture Notes in Computer Science, Cham: Springer Nature Switzerland, 2024, pp. 215–230, ISBN: 978-3-031-61056-1 978-3-031-61057-8. doi: 10.1007/978-3-031-61057-8_13. [Online]. Available: https://link.springer.com/10.1007/978-3-031-61057-8_13 (visited on 10/27/2025).
- [9] M. A. Ali, M. Dumas, and F. Milani, *Enhancing the accuracy of predictors of activity sequences of business processes*, Dec. 9, 2023. doi: 10.48550/arXiv.2312.05560. arXiv: 2312.05560[cs]. [Online]. Available: <http://arxiv.org/abs/2312.05560> (visited on 10/27/2025).
- [10] M. Van Luijken, I. Ketykó, and F. Mannhardt, “An experiment on transfer learning for suffix prediction on event logs,” in *Business Process Management Workshops*, J. De Weerd and L. Pufahl, Eds., vol. 492, Series Title: Lecture Notes in Business Information Processing, Cham: Springer Nature Switzerland, 2024, pp. 31–43, ISBN: 978-3-031-50973-5 978-3-031-50974-2. doi: 10.1007/978-3-031-50974-2_3. [Online]. Available: https://link.springer.com/10.1007/978-3-031-50974-2_3 (visited on 10/27/2025).
- [11] V. Pasquadibisceglie, A. Appice, and D. Malerba, “LUPIN: A LLM approach for activity suffix prediction in business process event logs,” in *2024 6th International Conference on Process Mining (ICPM)*, Kgs. Lyngby, Denmark: IEEE, Oct. 14, 2024, pp. 1–8, ISBN: 979-8-3503-6503-0. doi: 10.1109/ICPM63005.2024.10680620. [Online]. Available: <https://ieeexplore.ieee.org/document/10680620/> (visited on 10/27/2025).
- [12] L. Lin, L. Wen, and J. Wang, “MM-pred: A deep predictive model for multi-attribute event sequence,” in *Proceedings of the 2019 SIAM International Conference on Data Mining (SDM)*, _eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611975673.14>, pp. 118–126.

- doi: 10.1137/1.9781611975673.14. [Online]. Available: <https://epubs.siam.org/doi/abs/10.1137/1.9781611975673.14>.
- [13] C. Zheng, J. Wang, C. Wang, and B. Li, “A two-stage predictive process monitoring approach with enabled state filtering for variable suffix scenarios,” in *2025 IEEE International Conference on Web Services (ICWS)*, Helsinki, Finland: IEEE, Jul. 7, 2025, pp. 804–813, ISBN: 979-8-3315-5563-4. doi: 10.1109/ICWS67624.2025.00104. [Online]. Available: <https://ieeexplore.ieee.org/document/11169633/> (visited on 10/27/2025).
- [14] B. Wuyts, S. Vanden Broucke, and J. De Weerd, “CaLenDiR: Mitigating case-length distortion in deep-learning-based predictive process monitoring,” in *Process Mining Workshops*, A. Delgado and T. Slaats, Eds., vol. 533, Series Title: Lecture Notes in Business Information Processing, Cham: Springer Nature Switzerland, 2025, pp. 253–266, ISBN: 978-3-031-82224-7 978-3-031-82225-4. doi: 10.1007/978-3-031-82225-4_19. [Online]. Available: https://link.springer.com/10.1007/978-3-031-82225-4_19 (visited on 10/27/2025).
- [15] R. Graziosi, M. Ronzani, A. Buliga, *et al.*, *Generating multiperspective process traces using conditional variational autoencoders*, Mar. 26, 2025. doi: 10.21203/rs.3.rs-5653604/v1. [Online]. Available: <https://www.researchsquare.com/article/rs-5653604/v1> (visited on 10/27/2025).
- [16] A. Chiorrini, C. Diamantini, A. Mircoli, and D. Potena, “A preliminary study on the application of reinforcement learning for predictive process monitoring,” in *Process Mining Workshops*, S. Leemans and H. Leopold, Eds., vol. 406, Series Title: Lecture Notes in Business Information Processing, Cham: Springer International Publishing, 2021, pp. 124–135, ISBN: 978-3-030-72692-8 978-3-030-72693-5. doi: 10.1007/978-3-030-72693-5_10. [Online]. Available: https://link.springer.com/10.1007/978-3-030-72693-5_10 (visited on 10/27/2025).
- [17] B. Dalmás, F. Baranski, and D. Cortinovis, “Predicting process activities and timestamps with entity-embeddings neural networks,” in *Research Challenges in Information Science*, S. Cherfi, A. Perini, and S. Nurcan, Eds., vol. 415, Series Title: Lecture Notes in Business Information Processing, Cham: Springer International Publishing, 2021, pp. 393–408, ISBN: 978-3-030-75017-6 978-3-030-75018-3. doi: 10.1007/978-3-030-75018-3_26.

- [Online]. Available: https://link.springer.com/10.1007/978-3-030-75018-3_26 (visited on 10/27/2025).
- [18] F. Taymouri, M. L. Rosa, S. Erfani, Z. D. Bozorgi, and I. Verenich, *Predictive business process monitoring via generative adversarial nets: The case of next event prediction*, Apr. 1, 2020. doi: 10.48550/arXiv.2003.11268. arXiv: 2003.11268 [cs]. [Online]. Available: <http://arxiv.org/abs/2003.11268> (visited on 10/27/2025).
- [19] M. Camargo, M. Dumas, and O. González-Rojas, "Learning accurate LSTM models of business processes," in *Business Process Management*, T. Hildebrandt, B. F. Van Dongen, M. Röglinger, and J. Mendling, Eds., vol. 11675, Series Title: Lecture Notes in Computer Science, Cham: Springer International Publishing, 2019, pp. 286–302, ISBN: 978-3-030-26618-9 978-3-030-26619-6. doi: 10.1007/978-3-030-26619-6_19. [Online]. Available: https://link.springer.com/10.1007/978-3-030-26619-6_19 (visited on 10/27/2025).
- [20] N. Tax, I. Verenich, M. L. Rosa, and M. Dumas, "Predictive business process monitoring with LSTM neural networks," in vol. 10253, 2017, pp. 477–492. doi: 10.1007/978-3-319-59536-8_30. arXiv: 1612.02130 [stat]. [Online]. Available: <http://arxiv.org/abs/1612.02130> (visited on 10/27/2025).
- [21] H. Mustroph, M. Kunkler, and S. Rinderle-Ma, *An uncertainty-aware ED-LSTM for probabilistic suffix prediction*, Version Number: 2, 2025. doi: 10.48550/ARXIV.2505.21339. [Online]. Available: <https://arxiv.org/abs/2505.21339> (visited on 05/11/2026).
- [22] N. Mehdiyev, M. Majlatow, and P. Fettke, "Augmenting post-hoc explanations for predictive process monitoring with uncertainty quantification via conformalized monte carlo dropout," *Data & Knowledge Engineering*, vol. 156, p. 102402, Mar. 2025, ISSN: 0169023X. doi: 10.1016/j.datak.2024.102402. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0169023X24001265> (visited on 11/12/2025).
- [23] M. Majlatow, F. A. Shakil, A. Emrich, and N. Mehdiyev, "Uncertainty-aware predictive process monitoring in healthcare: Explainable insights into probability calibration for conformal prediction," *Applied Sciences*, vol. 15, no. 14, p. 7925, Jul. 16, 2025, ISSN: 2076-3417. doi: 10.3390/app15147925. [Online]. Available: <https://www.mdpi.com/2076-3417/15/14/7925> (visited on 11/12/2025).

- [24] N. Mehdiyev and P. Fettke, “Local post-hoc explanations for predictive process monitoring in manufacturing,” 2020, Version Number: 2. doi: 10.48550/ARXIV.2009.10513. [Online]. Available: <https://arxiv.org/abs/2009.10513> (visited on 10/29/2025).
- [25] B. Wickramanayake, Z. He, C. Ouyang, C. Moreira, Y. Xu, and R. Sindhgatta, “Building interpretable models for business process prediction using shared and specialised attention mechanisms,” *Knowledge-Based Systems*, vol. 248, p. 108 773, Jul. 2022, issn: 09507051. doi: 10.1016/j.knosys.2022.108773. arXiv: 2109.01419[cs]. [Online]. Available: <http://arxiv.org/abs/2109.01419> (visited on 11/12/2025).
- [26] B. Wu, S. Hong, and R. Lin, “Explainable prediction for business process activity with transformer neural networks,” *Knowledge and Information Systems*, vol. 67, no. 10, pp. 9221–9252, Oct. 2025, issn: 0219-1377, 0219-3116. doi: 10.1007/s10115-025-02501-7. [Online]. Available: <https://link.springer.com/10.1007/s10115-025-02501-7> (visited on 11/12/2025).
- [27] M. Käppel, L. Ackermann, S. Jablonski, and S. Härtl, *Attention please: What transformer models really learn for process prediction*, Aug. 12, 2024. doi: 10.48550/arXiv.2408.07097. arXiv: 2408.07097[cs]. [Online]. Available: <http://arxiv.org/abs/2408.07097> (visited on 10/29/2025).
- [28] M. Käppel, L. Ackermann, S. Jablonski, and S. Härtl, “Explaining transformer-based next activity prediction by using attention scores,” *Process Science*, vol. 2, no. 1, p. 11, Jun. 4, 2025, issn: 2948-2178. doi: 10.1007/s44311-025-00018-4. [Online]. Available: <https://link.springer.com/10.1007/s44311-025-00018-4> (visited on 10/29/2025).
- [29] T.-H. Huang, A. Metzger, and K. Pohl, “Counterfactual explanations for predictive business process monitoring,” in vol. 437, 2022, pp. 399–413. doi: 10.1007/978-3-030-95947-0_28. arXiv: 2202.12018[cs]. [Online]. Available: <http://arxiv.org/abs/2202.12018> (visited on 10/28/2025).
- [30] A. Stevens, C. Ouyang, J. D. Smedt, and C. Moreira, *Generating feasible and plausible counterfactual explanations for outcome prediction of business processes*, Mar. 14, 2024. doi: 10.48550/arXiv.2403.09232. arXiv: 2403.09232[cs]. [Online]. Available: <http://arxiv.org/abs/2403.09232> (visited on 10/06/2025).

- [31] A. Buliga, M. Vazifehdoostirani, L. Genga, *et al.*, “Uncovering patterns for local explanations in outcome-based predictive process monitoring,” in *Business Process Management*, A. Marrella, M. Resinas, M. Jans, and M. Rosemann, Eds., vol. 14940, Series Title: Lecture Notes in Computer Science, Cham: Springer Nature Switzerland, 2024, pp. 363–380, ISBN: 978-3-031-70395-9 978-3-031-70396-6. doi: 10.1007/978-3-031-70396-6_21. [Online]. Available: https://link.springer.com/10.1007/978-3-031-70396-6_21 (visited on 10/06/2025).
- [32] M. Vazifehdoostirani, A. Buliga, L. Genga, *et al.*, “Outcome-oriented local explanation using context-aware process patterns,” *Engineering Applications of Artificial Intelligence*, vol. 159, p. 111 387, Nov. 2025, ISSN: 09521976. doi: 10.1016/j.engappai.2025.111387. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0952197625013892> (visited on 11/12/2025).
- [33] A. Maita, M. Fantinato, S. Peres, and F. Maggi, “Towards a business-oriented approach to visualization-supported interpretability of prediction results in process mining,” in *Proceedings of the 25th International Conference on Enterprise Information Systems*, Prague, Czech Republic: SCITEPRESS - Science and Technology Publications, 2023, pp. 395–406, ISBN: 978-989-758-648-4. doi: 10.5220/0011976000003467. [Online]. Available: <https://www.scitepress.org/DigitalLibrary/Link.aspx?doi=10.5220/0011976000003467> (visited on 11/12/2025).
- [34] N. Mehdiyev, M. Majlatow, and P. Fettke, *Communicating uncertainty in machine learning explanations: A visualization analytics approach for predictive process monitoring*, Apr. 12, 2023. doi: 10.48550/arXiv.2304.05736. arXiv: 2304.05736 [cs]. [Online]. Available: <http://arxiv.org/abs/2304.05736> (visited on 04/24/2026).
- [35] S. Chae, S. Lee, H. Hauptmann, H. A. Reijers, and X. Lu, *The role of explanation styles and perceived accuracy on decision making in predictive process monitoring*, Jun. 19, 2025. doi: 10.48550/arXiv.2506.16617. arXiv: 2506.16617 [cs]. [Online]. Available: <http://arxiv.org/abs/2506.16617> (visited on 10/06/2025).
- [36] N. Mehdiyev and P. Fettke, “Explainable artificial intelligence for process mining: A general overview and application of a novel local explanation approach for predictive process monitoring,” in *Interpretable Artificial Intelligence: A Perspective of Granular Computing*, W. Pedrycz and S.-M. Chen, Eds., vol. 937, Series Title: Studies in Computational Intelli-

- gence, Cham: Springer International Publishing, 2021, pp. 1–28, ISBN: 978-3-030-64948-7 978-3-030-64949-4. DOI: 10.1007/978-3-030-64949-4_1. [Online]. Available: http://link.springer.com/10.1007/978-3-030-64949-4_1 (visited on 11/12/2025).
- [37] S. Lundberg and S.-I. Lee, *A unified approach to interpreting model predictions*, Nov. 25, 2017. DOI: 10.48550/arXiv.1705.07874. arXiv: 1705.07874[cs]. [Online]. Available: <http://arxiv.org/abs/1705.07874> (visited on 10/29/2025).
- [38] E. Štrumbelj and I. Kononenko, “Explaining prediction models and individual predictions with feature contributions,” *Knowledge and Information Systems*, vol. 41, no. 3, pp. 647–665, Dec. 2014, ISSN: 0219-1377, 0219-3116. DOI: 10.1007/s10115-013-0679-x. [Online]. Available: <http://link.springer.com/10.1007/s10115-013-0679-x> (visited on 05/13/2026).
- [39] S. Jain and B. C. Wallace, *Attention is not explanation*, May 8, 2019. DOI: 10.48550/arXiv.1902.10186. arXiv: 1902.10186[cs]. [Online]. Available: <http://arxiv.org/abs/1902.10186> (visited on 04/24/2026).
- [40] S. Wiegrefe and Y. Pinter, *Attention is not not explanation*, Sep. 5, 2019. DOI: 10.48550/arXiv.1908.04626. arXiv: 1908.04626[cs]. [Online]. Available: <http://arxiv.org/abs/1908.04626> (visited on 04/24/2026).
- [41] M. Sundararajan, A. Taly, and Q. Yan, *Axiomatic attribution for deep networks*, Jun. 13, 2017. DOI: 10.48550/arXiv.1703.01365. arXiv: 1703.01365[cs]. [Online]. Available: <http://arxiv.org/abs/1703.01365> (visited on 04/08/2026).
- [42] S. Weinzierl, S. Zilker, J. Brunk, K. Revoredo, M. Matzner, and J. Becker, “XNAP: Making LSTM-based next activity predictions explainable by using LRP,” in vol. 397, 2020, pp. 129–141. DOI: 10.1007/978-3-030-66498-5_10. arXiv: 2008.07993[cs]. [Online]. Available: <http://arxiv.org/abs/2008.07993> (visited on 04/24/2026).
- [43] W. M. P. Van Der Aalst and J. Carmona, Eds., *Process Mining Handbook*, vol. 448, Lecture Notes in Business Information Processing, Cham: Springer International Publishing, 2022, ISBN: 978-3-031-08847-6 978-3-031-08848-3. DOI: 10.1007/978-3-031-08848-3. [Online]. Available: <https://link.springer.com/10.1007/978-3-031-08848-3> (visited on 10/21/2025).

- [44] R. Galanti, B. Coma-Puig, M. d. Leoni, J. Carmona, and N. Navarin, *Explainable predictive process monitoring*, Sep. 16, 2020. doi: 10.48550/arXiv.2008.01807. arXiv: 2008.01807[cs]. [Online]. Available: <http://arxiv.org/abs/2008.01807> (visited on 10/06/2025).
- [45] A. Stevens, J. De Smedt, and J. Peeperkorn, “Quantifying explainability in outcome-oriented predictive process monitoring,” in *Process Mining Workshops*, J. Munoz-Gama and X. Lu, Eds., vol. 433, Series Title: Lecture Notes in Business Information Processing, Cham: Springer International Publishing, 2022, pp. 194–206, ISBN: 978-3-030-98580-6 978-3-030-98581-3. doi: 10.1007/978-3-030-98581-3_15. [Online]. Available: https://link.springer.com/10.1007/978-3-030-98581-3_15 (visited on 10/06/2025).
- [46] A. Stevens and J. De Smedt, “Explainability in process outcome prediction: Guidelines to obtain interpretable and faithful models,” *European Journal of Operational Research*, vol. 317, no. 2, pp. 317–329, Sep. 2024, ISSN: 03772217. doi: 10.1016/j.ejor.2023.09.010. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0377221723007117> (visited on 10/06/2025).
- [47] M. T. Ribeiro, S. Singh, and C. Guestrin, “*why should i trust you?*”: *Explaining the predictions of any classifier*, Aug. 9, 2016. doi: 10.48550/arXiv.1602.04938. arXiv: 1602.04938[cs]. [Online]. Available: <http://arxiv.org/abs/1602.04938> (visited on 10/29/2025).
- [48] D. Gunning and D. W. Aha, “DARPA’s explainable artificial intelligence program,” *AI Magazine*, vol. 40, no. 2, pp. 44–58, Jun. 2019, ISSN: 0738-4602, 2371-9621. doi: 10.1609/aimag.v40i2.2850. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1609/aimag.v40i2.2850> (visited on 09/29/2025).
- [49] R. Geirhos, J.-H. Jacobsen, C. Michaelis, *et al.*, “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, no. 11, pp. 665–673, Nov. 10, 2020, ISSN: 2522-5839. doi: 10.1038/s42256-020-00257-z. arXiv: 2004.07780[cs]. [Online]. Available: <http://arxiv.org/abs/2004.07780> (visited on 04/27/2026).
- [50] S. Lapuschkin, S. Wäldchen, A. Binder, G. Montavon, W. Samek, and K.-R. Müller, “Unmasking clever hans predictors and assessing what machines really learn,” *Nature Communications*, vol. 10, no. 1, p. 1096, Mar. 11, 2019, ISSN: 2041-1723. doi: 10.1038/s41467-019-

- 08987-4. [Online]. Available: <https://www.nature.com/articles/s41467-019-08987-4> (visited on 04/27/2026).
- [51] A. J. DeGrave, J. D. Janizek, and S.-I. Lee, “AI for radiographic COVID-19 detection selects shortcuts over signal,” *Nature Machine Intelligence*, vol. 3, no. 7, pp. 610–619, May 31, 2021, issn: 2522-5839. doi: 10.1038/s42256-021-00338-7. [Online]. Available: <https://www.nature.com/articles/s42256-021-00338-7> (visited on 04/27/2026).
- [52] A. Rozinat and W. v. d. Aalst, “Decision mining in ProM,” pp. 420–425, 2006.
- [53] M. De Leoni and W. M. P. Van Der Aalst, “Data-aware process mining: Discovering decisions in processes using alignments,” in *Proceedings of the 28th Annual ACM Symposium on Applied Computing*, Coimbra Portugal: ACM, Mar. 18, 2013, pp. 1454–1461, isbn: 978-1-4503-1656-9. doi: 10.1145/2480362.2480633. [Online]. Available: <https://dl.acm.org/doi/10.1145/2480362.2480633> (visited on 04/12/2026).
- [54] M. Song, C. W. Günther, and W. M. P. Van Der Aalst, “Trace clustering in process mining,” in *Business Process Management Workshops*, D. Ardagna, M. Mecella, and J. Yang, Eds., vol. 17, Series Title: Lecture Notes in Business Information Processing, Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 109–120, isbn: 978-3-642-00327-1 978-3-642-00328-8. doi: 10.1007/978-3-642-00328-8_11. [Online]. Available: http://link.springer.com/10.1007/978-3-642-00328-8_11 (visited on 05/07/2026).
- [55] M. Craven and J. W. Shavlik, “Extracting tree-structured representations of trained networks,” 1995.
- [56] F. Mannhardt, M. De Leoni, H. A. Reijers, and W. M. P. Van Der Aalst, “Decision mining revisited - discovering overlapping rules,” in *Advanced Information Systems Engineering*, S. Nurcan, P. Soffer, M. Bajec, and J. Eder, Eds., vol. 9694, Series Title: Lecture Notes in Computer Science, Cham: Springer International Publishing, 2016, pp. 377–392, isbn: 978-3-319-39695-8 978-3-319-39696-5. doi: 10.1007/978-3-319-39696-5_23. [Online]. Available: https://link.springer.com/10.1007/978-3-319-39696-5_23 (visited on 04/12/2026).

A Appendix

Table 8*Engineered features used by the per-gateway surrogate decision trees (BPIC17)*

Feature	Type	Source	Description
<i>Prefix structure</i>			
prefix_length	numeric	derived	Number of non-padding events in the prefix.
<i>Case-level categorical attributes (value at the gateway event)</i>			
last_case:LoanGoal	categorical	case:LoanGoal	Stated purpose of the loan application.
last_case:ApplicationType	categorical	case:ApplicationType	Type of loan application submitted.
<i>Case-level numerical attributes (value at the gateway event)</i>			
last_case:RequestedAmount	numeric	case:RequestedAmount	Loan amount requested by the applicant.
last_FirstWithdrawalAmount	numeric	FirstWithdrawalAmount	Amount of the first withdrawal on the loan.
last_NumberOfTerms	numeric	NumberOfTerms	Number of repayment terms of the loan.
last_MonthlyCost	numeric	MonthlyCost	Monthly repayment cost associated with the loan.
last_CreditScore	numeric	CreditScore	Credit score recorded for the applicant.
<i>Positional windows of event-level categorical attributes ($w = 40$, $pad = \langle pad \rangle$)</i>			
concept:name_pos_-1,..., concept:name_pos_-40	categorical	concept:name	Activity label at offset $-w$ from the gateway, for $w = 1, \dots, 40$. Positions beyond the prefix start are set to $\langle pad \rangle$.
org:resource_pos_-1,..., org:resource_pos_-40	categorical	org:resource	Resource at offset $-w$ from the gateway.
lifecycle:transition_pos_-1,..., lifecycle:transition_pos_-40	categorical	lifecycle:transition	Lifecycle transition at offset $-w$ from the gateway.
Action_pos_-1,..., Action_pos_-40	categorical	Action	Action label at offset $-w$ from the gateway.
EventOrigin_pos_-1,..., EventOrigin_pos_-40	categorical	EventOrigin	Event origin at offset $-w$ from the gateway.
Accepted_pos_-1,..., Accepted_pos_-40	categorical	Accepted	Acceptance indicator at offset $-w$ from the gateway.
Selected_pos_-1,..., Selected_pos_-40	categorical	Selected	Selection indicator at offset $-w$ from the gateway.
<i>Positional windows of event-level numerical attributes ($w = 40$, $pad = NaN$)</i>			
case_elapsed_time_pos_-1,..., case_elapsed_time_pos_-40	numeric	case_elapsed_time	Elapsed time since case start at offset $-w$ from the gateway, for $w = 1, \dots, 40$. Positions beyond the prefix start are set to NaN and treated as missing by the C4.5 learner.
event_elapsed_time_pos_-1,..., event_elapsed_time_pos_-40	numeric	event_elapsed_time	Duration of the event at offset $-w$ from the gateway.
day_in_week_pos_-1,..., day_in_week_pos_-40	numeric	day_in_week	Day-of-week indicator at offset $-w$ from the gateway.
seconds_in_day_pos_-1,..., seconds_in_day_pos_-40	numeric	seconds_in_day	Seconds elapsed within the calendar day at offset $-w$ from the gateway.

Table 9*Engineered features used by the per-gateway surrogate decision trees (Helpdesk)*

Feature	Type	Source	Description
<i>Prefix structure</i>			
prefix_length	numeric	derived	Number of non-padding events in the prefix.
<i>Case-level categorical attributes (value at the gateway event)</i>			
last_Variant_index	categorical	Variant_index	Case variant identifier at the gateway event.
last_seriousness	categorical	seriousness	Ticket seriousness level.
last_customer	categorical	customer	Customer identifier.
last_product	categorical	product	Product associated with the ticket.
last_responsible_section	categorical	responsible_section	Section responsible for the ticket.
last_seriousness_2	categorical	seriousness_2	Secondary seriousness classification.
last_service_level	categorical	service_level	Agreed service level.
last_service_type	categorical	service_type	Type of service requested.
last_support_section	categorical	support_section	Support section currently handling the ticket.
last_workgroup	categorical	workgroup	Workgroup currently handling the ticket.
<i>Positional windows of event-level categorical attributes ($w = 10$, $pad = \langle pad \rangle$)</i>			
Activity_pos_-1,..., Activity_pos_-10	categorical	Activity	Activity label at offset $-w$ from the gateway, for $w = 1, \dots, 10$. Positions beyond the prefix start are set to $\langle pad \rangle$.
Resource_pos_-1,..., Resource_pos_-10	categorical	Resource	Resource at offset $-w$ from the gateway.
<i>Positional windows of event-level numerical attributes ($w = 10$, $pad = NaN$)</i>			
case_elapsed_time_pos_-1,..., case_elapsed_time_pos_-10	numeric	case_elapsed_time	Elapsed time since case start at offset $-w$ from the gateway, for $w = 1, \dots, 10$. Positions beyond the prefix start are set to NaN and treated as missing by the C4.5 learner.
event_elapsed_time_pos_-1,..., event_elapsed_time_pos_-10	numeric	event_elapsed_time	Duration of the event at offset $-w$ from the gateway.
day_in_week_pos_-1,..., day_in_week_pos_-10	numeric	day_in_week	Day-of-week indicator at offset $-w$ from the gateway.
seconds_in_day_pos_-1,..., seconds_in_day_pos_-10	numeric	seconds_in_day	Seconds elapsed within the calendar day at offset $-w$ from the gateway.